

Week 5

Neural Networks

Nebius Academy



Linear model

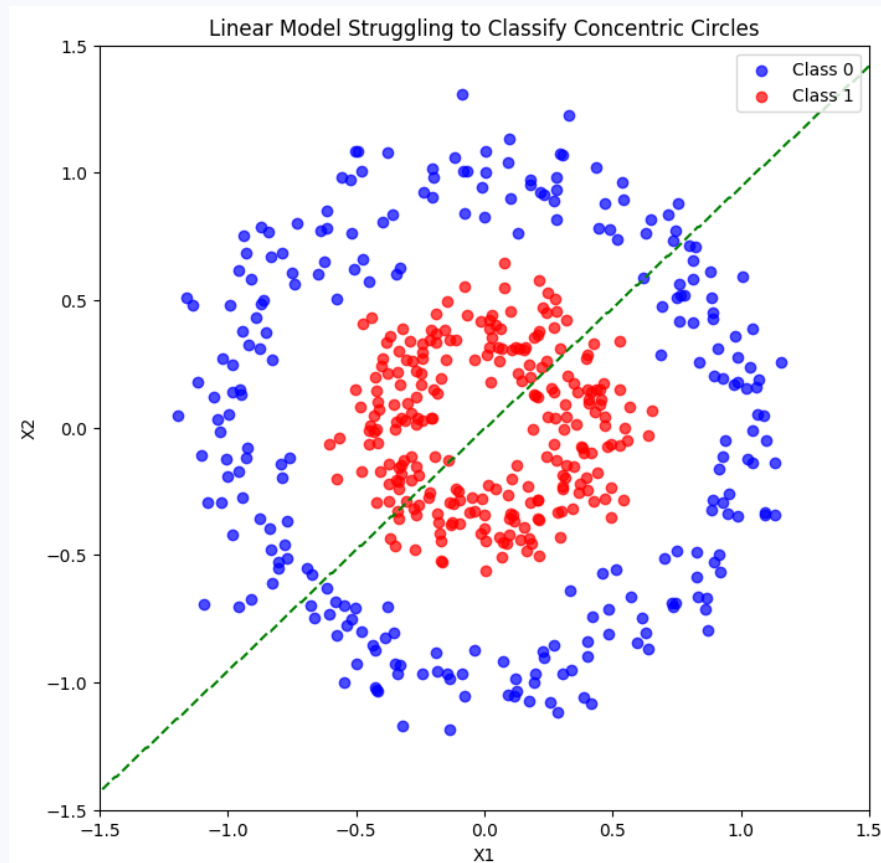


Neural Network

Feature expressivity matters

A linear model can't solve this classification task. At the same time, it can be solved by a linear model on advanced feature $x_1^2 + x_2^2$:

$$\text{class} = \text{sign}(x_1^2 + x_2^2 - 0.6^2)$$

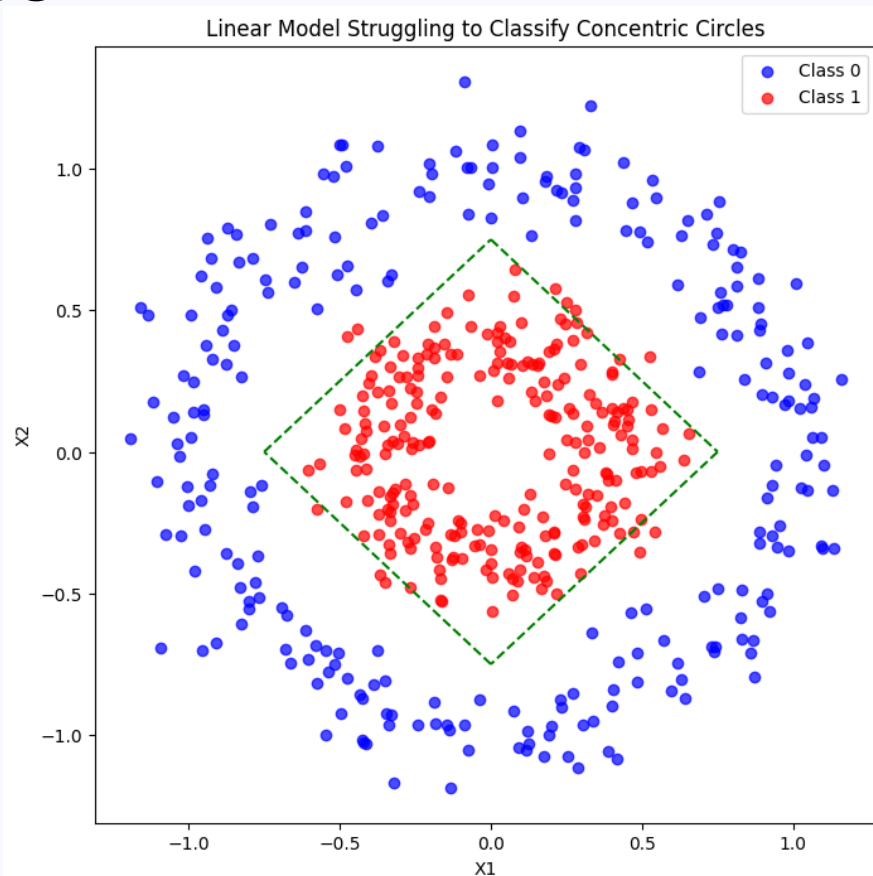


Feature expressivity matters

Or like this

$$\begin{aligned} \text{class} = & \text{sign}(\max(0, x_1 + x_2 - 0.75) \\ & + \max(0, x_1 - x_2 - 0.75) \\ & + \max(0, -x_1 - x_2 - 0.75) \\ & + \max(0, -x_1 + x_2 - 0.75)) \end{aligned}$$

Classic ML had a whole universe of feature engineering methods.



Automating feature engineering

Combinations of linear transformations and nonlinearity can do much:

$$(x_1, x_2) \rightarrow (x_1 + x_2 - 0.75, x_1 - x_2 - 0.75, \\ -x_1 - x_2 - 0.75, -x_1 + x_2 - 0.75)$$

Linear

$$\rightarrow (\max(0, x_1 + x_2 - 0.75), \max(0, x_1 - x_2 - 0.75), \\ \max(0, -x_1 - x_2 - 0.75), \max(0, -x_1 + x_2 - 0.75))$$

Nonlinearity

$$\rightarrow \max(0, x_1 + x_2 - 0.75) + \max(0, x_1 - x_2 - 0.75) \\ + \max(0, -x_1 - x_2 - 0.75) + \max(0, -x_1 + x_2 - 0.75)$$

Linear

Automating feature engineering

Combinations of linear transformations and nonlinearity can do much:

$$(x, y) \rightarrow (w_{10} + w_{11}x_1 + w_{12}x_2, w_{20} + w_{21}x_1 + w_{22}x_2, \\ w_{30} + w_{31}x_1 + w_{32}x_2, w_{40} + w_{41}x_1 + w_{42}x_2)$$

Linear

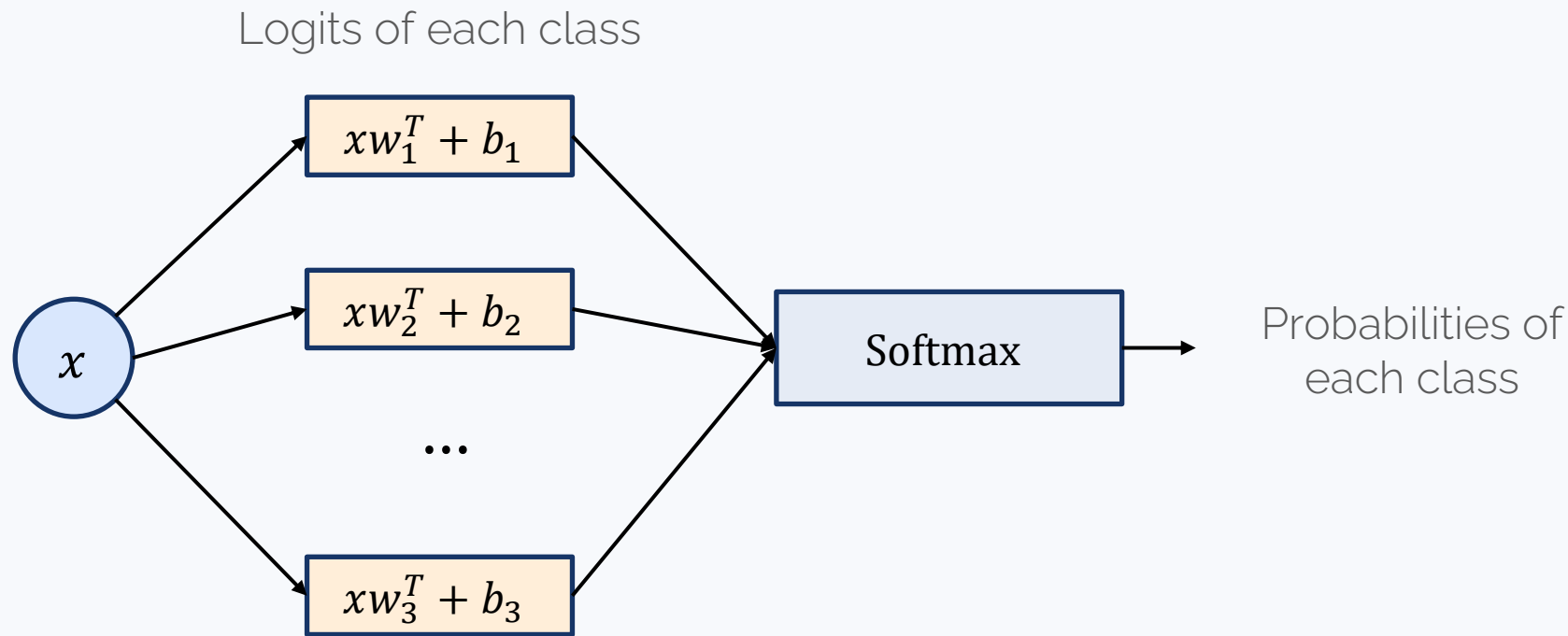
$$\rightarrow (\max(0, x'_1), \max(0, x'_2), \\ \max(0, x'_3), \max(0, x'_4))$$

Nonlinearity

$$\rightarrow u_1 \max(0, x'_1) + u_2 \max(0, x'_2) \\ + u_3 \max(0, x'_3) + u_4 \max(0, x'_4)$$

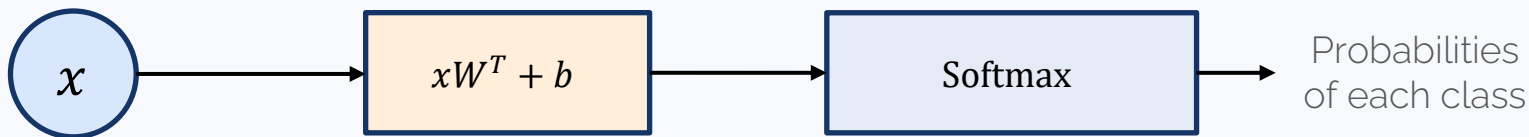
Linear

A model for multiclass classification



A model for multiclass classification

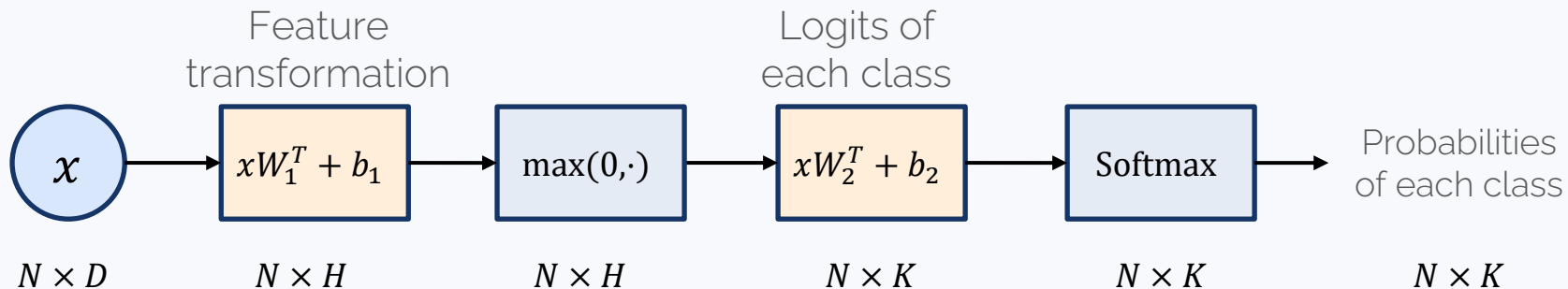
Logits of each class



$$\text{Logits} = x \cdot [w_1^T, \dots, w_K^T] = [x \cdot w_1^T, \dots, x \cdot w_K^T]$$

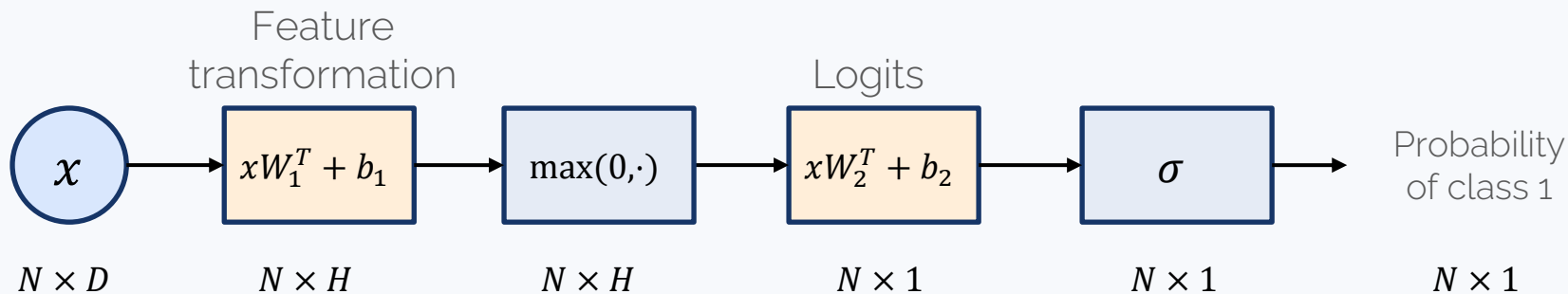
A diagram showing the matrix representation of the logit equation. On the left, a green horizontal rectangle represents the "Logits" vector. This is followed by an equals sign, then a blue horizontal rectangle representing the input vector x . This is followed by a dot product with a yellow box containing three columns labeled w_1^T , an ellipsis, and w_K^T . This is followed by another equals sign, then a large square bracket containing a sequence of terms: a blue rectangle, a dot, a yellow box with w_1^T , an ellipsis, another blue rectangle, a dot, and a yellow box with w_K^T .

A model for multiclass classification



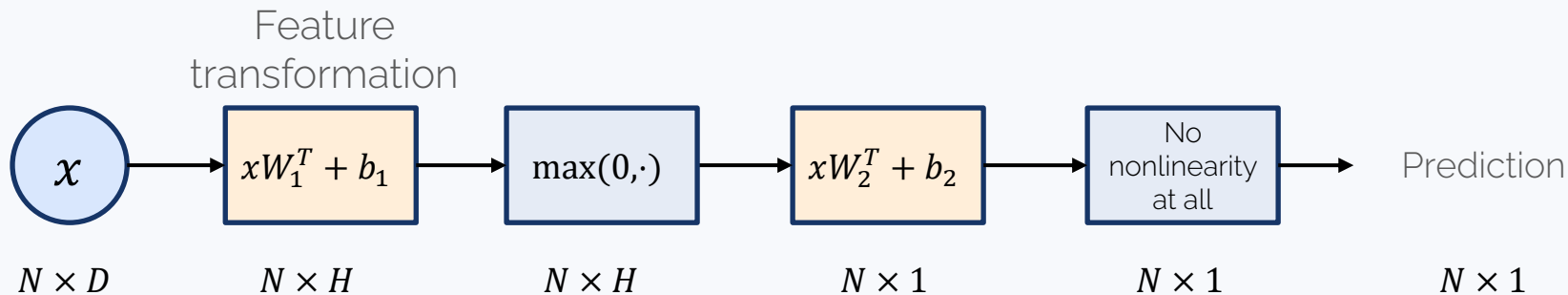
- W_1 has size $H \times D$
- W_2 has size $K \times H$

A model for binary classification



- W_1 has size $H \times D$
- W_2 has size $1 \times H$

A model for regression



- W_1 has size $H \times D$
- W_2 has size $1 \times H$

A **fully connected neural network (MLP)**

- Input
- Repeat:
 - **Linear layer** $x_m \mapsto x_m W_m^T + b_m$
 - **Nonlinearity**: σ , $\tanh$, $\text{ReLU}(x) = \max(0, x)$ etc
- Softmax/sigmoid/nothing -> output

Why nonlinearities are important?

Without nonlinearity:

$$\begin{aligned} \left(\left((xW_1^T + b_1)W_2^T + b_2 \right) \dots \right) W_M^T + b_M = \\ = xW_1^T W_2^T \dots W_M^T + bias \end{aligned}$$

Just a single linear model!

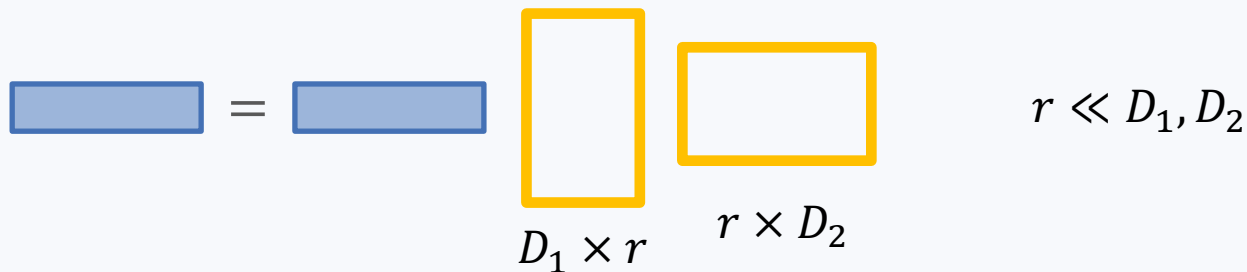
Why nonlinearities are important?

Without nonlinearity:

$$\begin{aligned} & \left(\left((xW_1^T + b_1)W_2^T + b_2 \right) \dots \right) W_M^T + b_M = \\ & = xW_1^T W_2^T \dots W_M^T + \text{bias} \end{aligned}$$

Just a single linear model!

It might be useful in one case (dimensionality reduction)

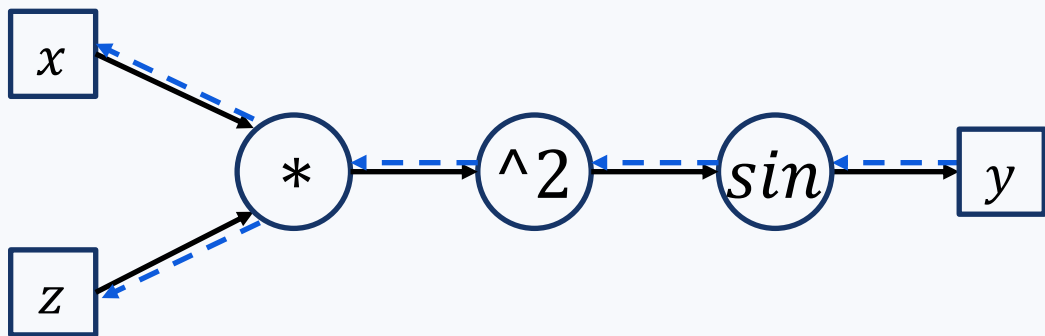


Error backpropagation

Automating differentiation

Every expression is a computational graph

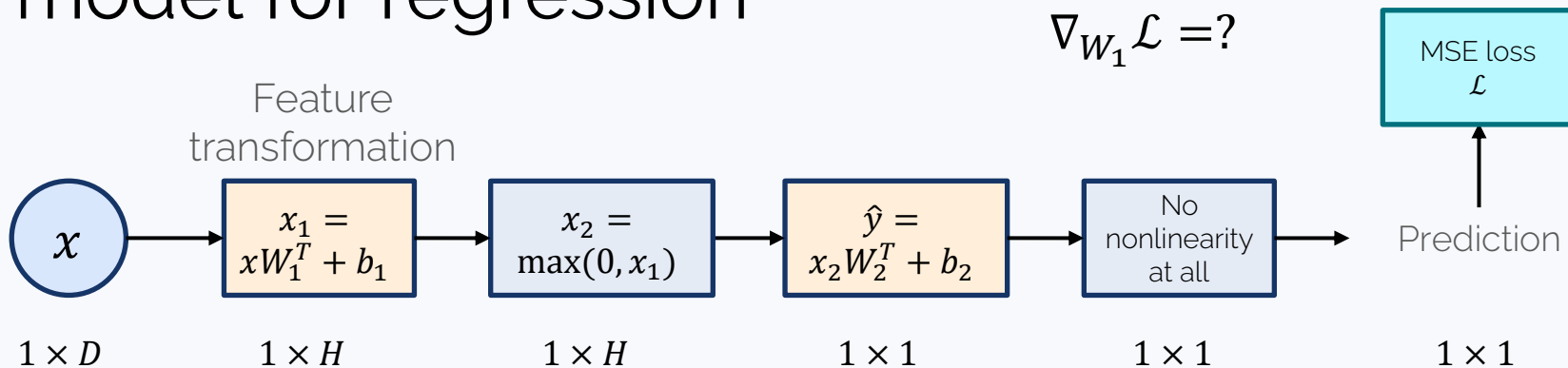
Example. $f(x, z) = \sin(x \cdot z)^2$



Derivatives flow backward

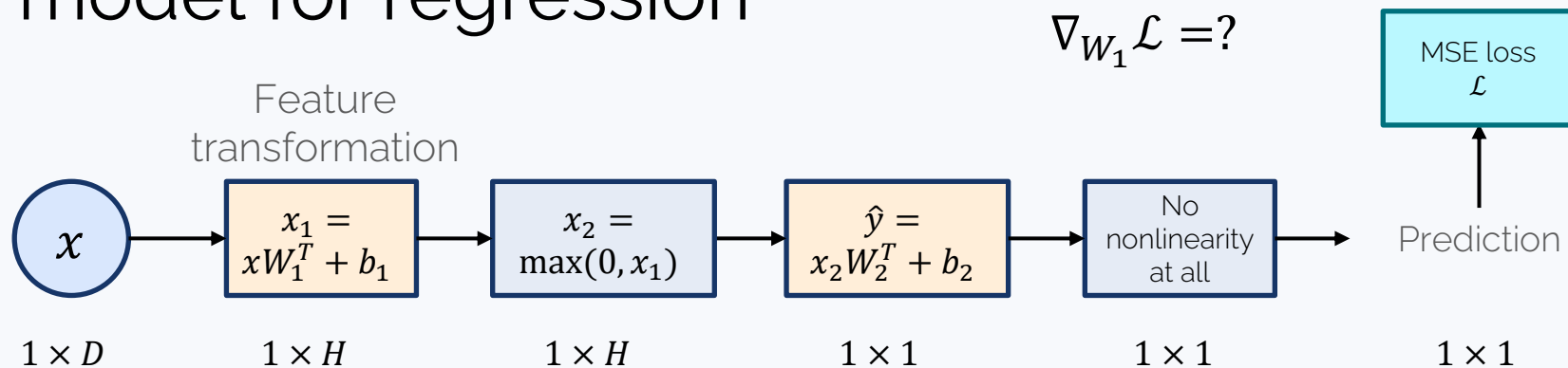
$$\begin{aligned} & \left. \frac{\partial}{\partial x} \sin(x \cdot z)^2 \right|_{\substack{x=1 \\ z=2}} \\ &= \cos \left((x \cdot z)^2 \right|_{\substack{x=1 \\ z=2}} \\ & \cdot 2 \left((x \cdot z) \right|_{\substack{x=1 \\ z=2}} \\ & \cdot z \end{aligned}$$

A model for regression



$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} [(\hat{y} - y)^2] = 2(\hat{y} - y)$$

A model for regression

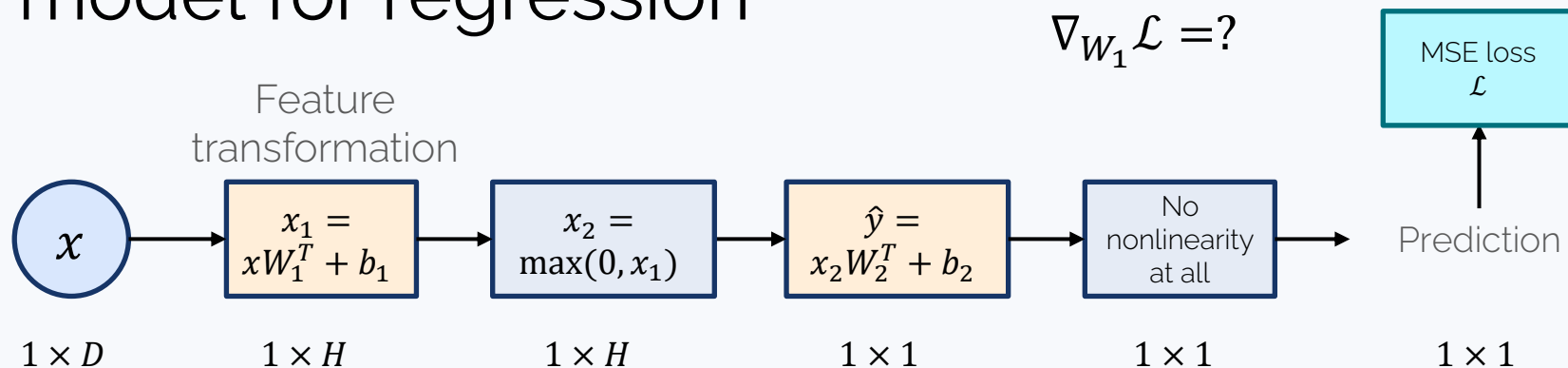


$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} [(\hat{y} - y)^2] = 2(\hat{y} - y)$$

$$\frac{\partial \mathcal{L}}{\partial x_{2i}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \bigg|_{\hat{y} = x_2 W_2^T + b_2} \frac{\partial \hat{y}}{\partial x_{2i}}$$

$$\begin{aligned} \frac{\partial \hat{y}}{\partial x_{2i}} &= \frac{\partial}{\partial x_{2i}} [x_2 W_2^T + b_2] = \\ &= \frac{\partial}{\partial x_{2i}} \left[\sum_j x_{2j} w_{2j} \right] = w_{2i} \end{aligned}$$

A model for regression



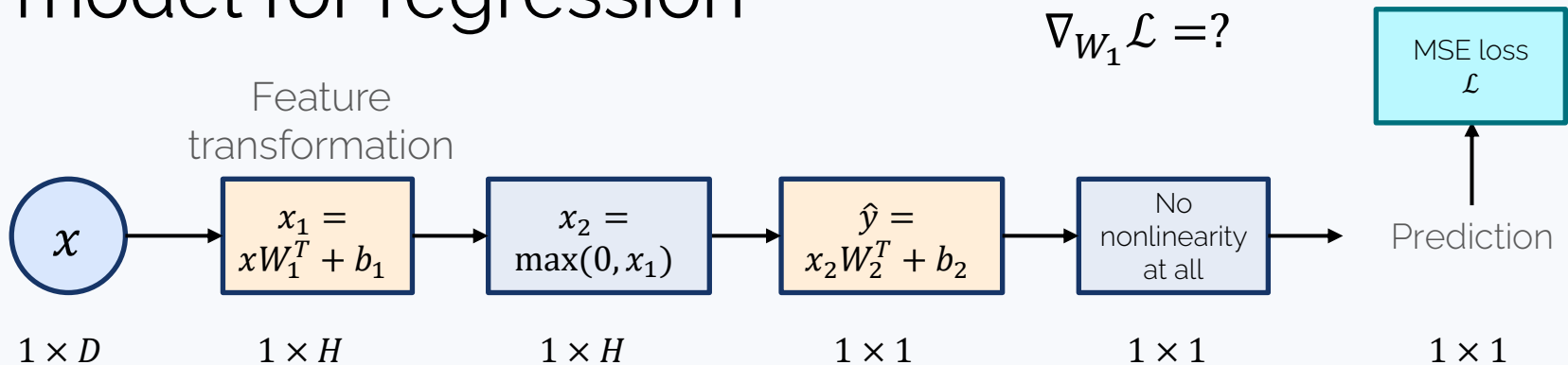
$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} [(\hat{y} - y)^2] = 2(\hat{y} - y)$$

$$\nabla_{x_2} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \hat{y}} W_2$$

$$\frac{\partial \mathcal{L}}{\partial x_{2i}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \bigg|_{\hat{y} = x_2 W_2^T + b_2} \frac{\partial \hat{y}}{\partial x_{2i}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} w_{2i}$$

$$\begin{aligned} \frac{\partial \hat{y}}{\partial x_{2i}} &= \frac{\partial}{\partial x_{2i}} [x_2 W_2^T + b_2] = \\ &= \frac{\partial}{\partial x_{2i}} \left[\sum_j x_{2j} w_{2j} \right] = w_{2i} \end{aligned}$$

A model for regression

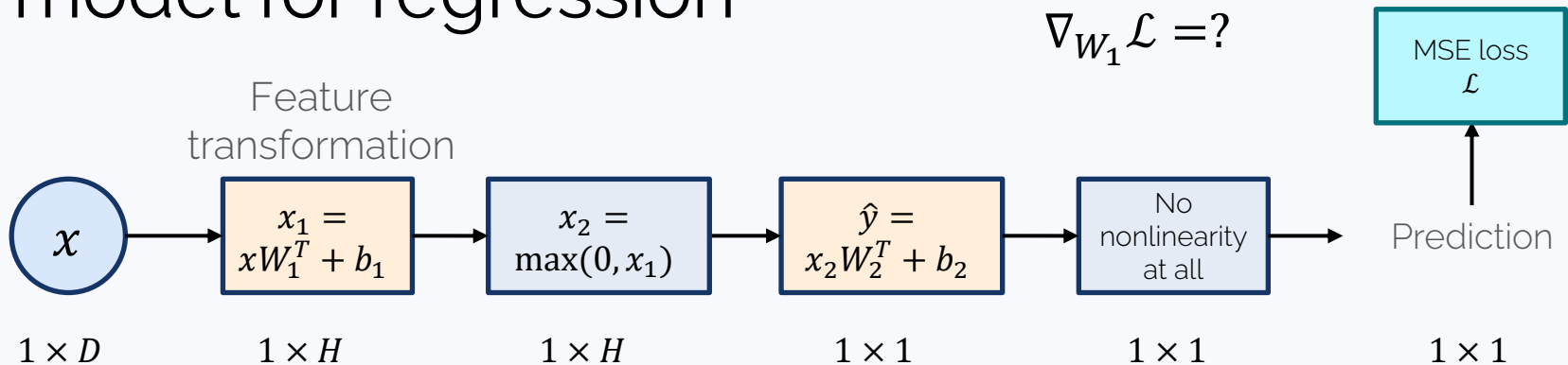


$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} [(\hat{y} - y)^2] = 2(\hat{y} - y)$$

$$\frac{\partial \mathcal{L}}{\partial x_{1i}} = \sum_j \frac{\partial \mathcal{L}}{\partial x_{2j}} \frac{\partial x_{2j}}{\partial x_{1i}} = \sum_j \frac{\partial \mathcal{L}}{\partial x_{2j}} \delta_{ij} H(x_{1i}) = \frac{\partial \mathcal{L}}{\partial x_{2i}} H(x_{1i})$$

$$\nabla_{x_2} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \hat{y}} W_2$$

A model for regression



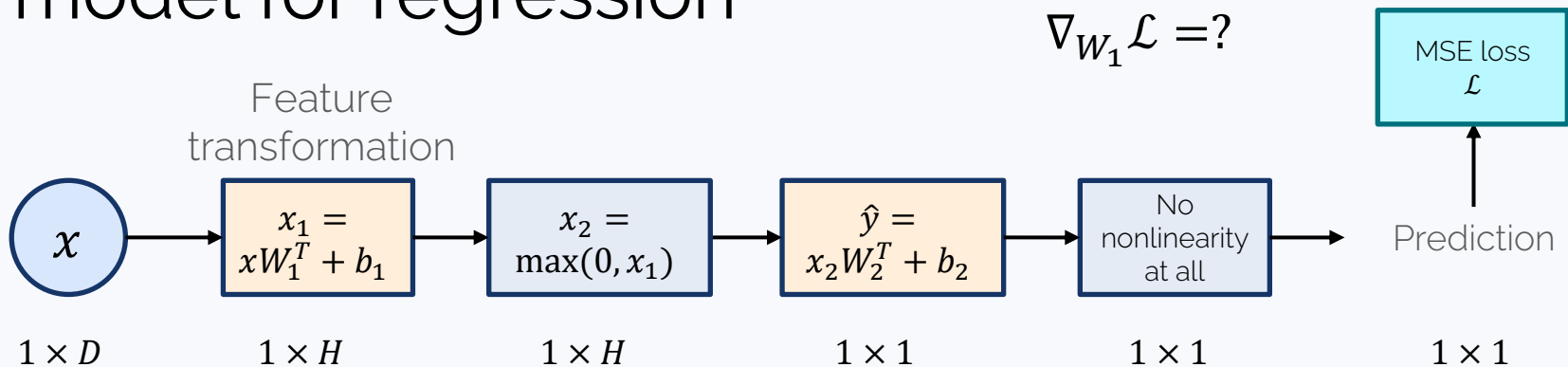
$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} [(\hat{y} - y)^2] = 2(\hat{y} - y)$$

$$\frac{\partial \mathcal{L}}{\partial x_{1i}} = \sum_j \frac{\partial \mathcal{L}}{\partial x_{2j}} \frac{\partial x_{2j}}{\partial x_{1i}} = \sum_j \frac{\partial \mathcal{L}}{\partial x_{2j}} \delta_{ij} H(x_{1i}) = \frac{\partial \mathcal{L}}{\partial x_{2i}} H(x_{1i})$$

$$\nabla_{x_2} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \hat{y}} W_2$$

$$\nabla_{x_1} \mathcal{L} = \nabla_{x_2} \mathcal{L} \odot H(x_1)$$

A model for regression

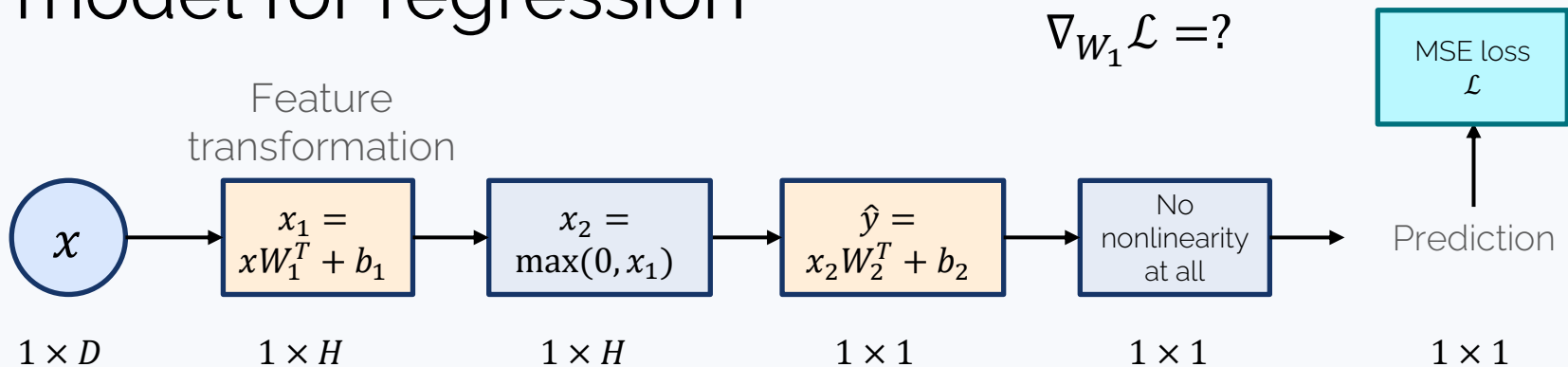


$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} [(\hat{y} - y)^2] = 2(\hat{y} - y)$$

$$\nabla_{x_2} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \hat{y}} W_2$$

$$\nabla_{x_1} \mathcal{L} = \nabla_{x_2} \mathcal{L} \odot H(x_1)$$

A model for regression



$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} [(\hat{y} - y)^2] = 2(\hat{y} - y)$$

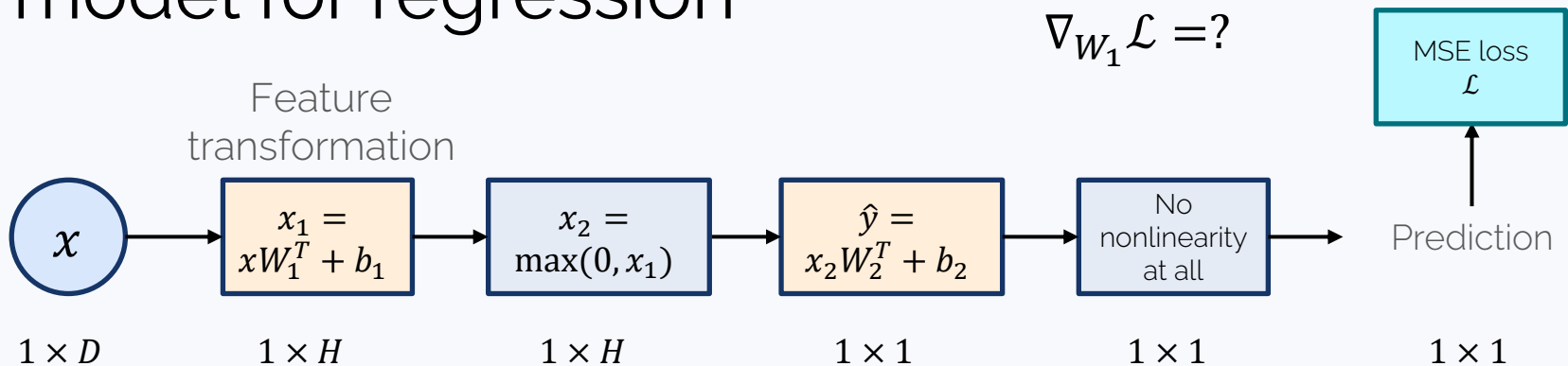
$$\frac{\partial \mathcal{L}}{\partial w_{1ij}} = \sum_k \frac{\partial \mathcal{L}}{\partial x_{1k}} \frac{\partial x_{1k}}{\partial w_{1ij}} = \sum_k \frac{\partial \mathcal{L}}{\partial x_{1k}} x_j \delta_{ik}$$

$$\nabla_{x_2} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \hat{y}} W_2$$

$$\nabla_{x_1} \mathcal{L} = \nabla_{x_2} \mathcal{L} \odot H(x_1)$$

$$\begin{aligned} \frac{\partial x_{1k}}{\partial w_{1ij}} &= \frac{\partial}{\partial w_{1ij}} [xW_1^T + b_1]_k = \\ &= \frac{\partial}{\partial w_{1ij}} \left[\sum_t x_t w_{1kt} \right] = x_j \delta_{ik} \end{aligned}$$

A model for regression



$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} [(\hat{y} - y)^2] = 2(\hat{y} - y)$$

$$\frac{\partial \mathcal{L}}{\partial w_{1ij}} = \sum_k \frac{\partial \mathcal{L}}{\partial x_{1k}} \frac{\partial x_{1k}}{\partial w_{1ij}} = \sum_k \frac{\partial \mathcal{L}}{\partial x_{1k}} x_j \delta_{ik} = \frac{\partial \mathcal{L}}{\partial x_{1i}} x_j$$

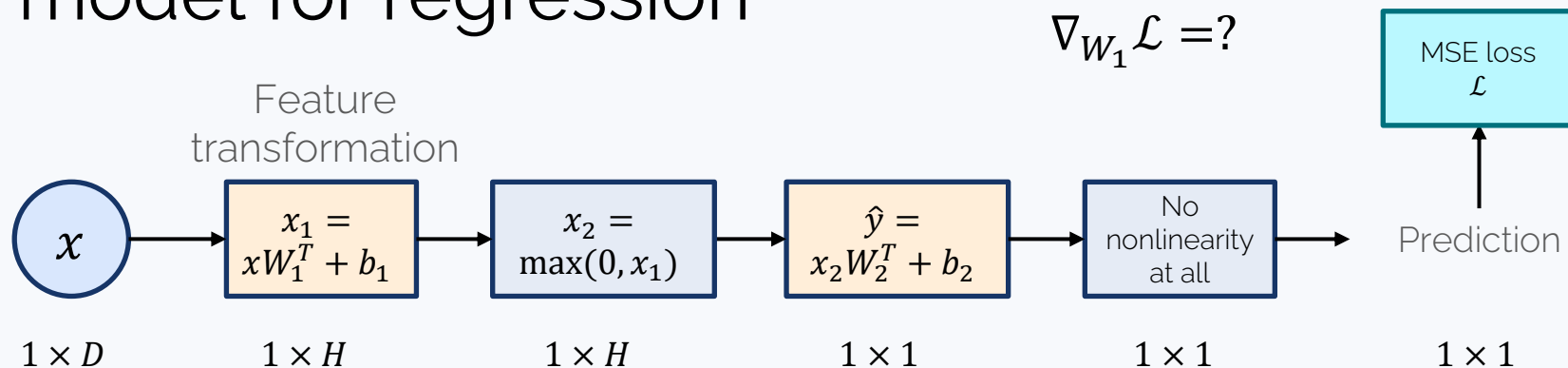
$$\nabla_{x_2} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \hat{y}} W_2$$

$$\nabla_{x_1} \mathcal{L} = \nabla_{x_2} \mathcal{L} \odot H(x_1)$$

$$\nabla_{W_1} \mathcal{L} = x^T \cdot \nabla_{x_1} \mathcal{L}$$

$$\begin{aligned} \frac{\partial x_{1k}}{\partial w_{1ij}} &= \frac{\partial}{\partial w_{1ij}} [xW_1^T + b_1]_k = \\ &= \frac{\partial}{\partial w_{1ij}} \left[\sum_t x_t w_{1kt} \right] = x_j \delta_{ik} \end{aligned}$$

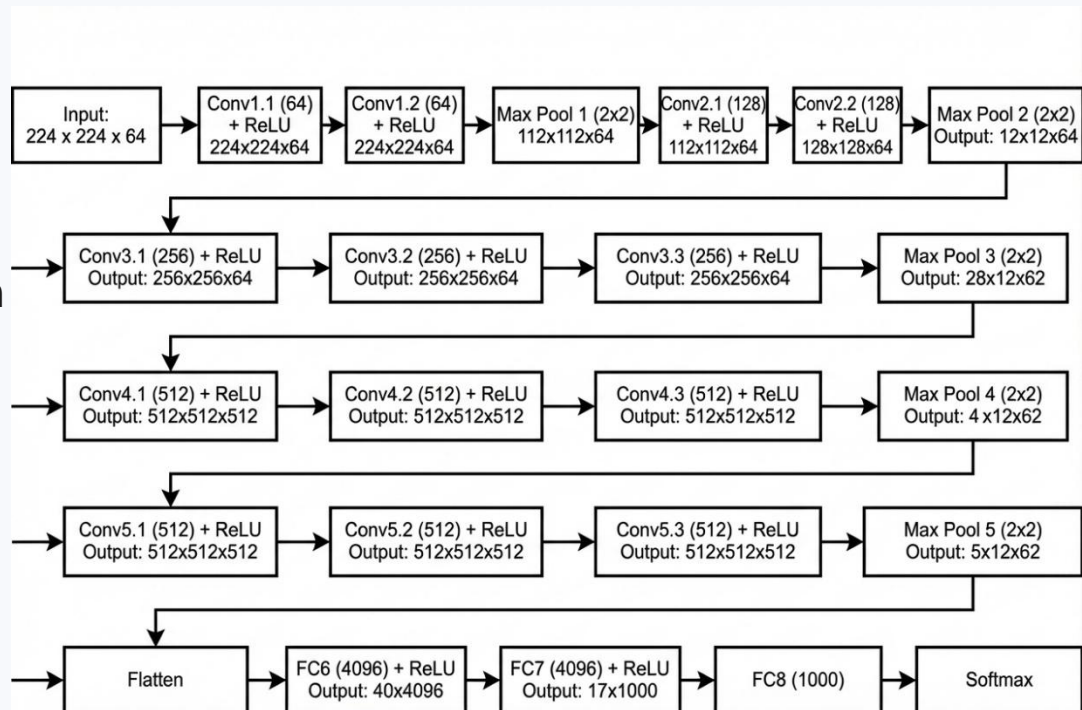
A model for regression



$$\begin{aligned}
 \nabla_{W_1} \mathcal{L} &= x^T \cdot \nabla_{x_1} \mathcal{L} & \nabla_{x_1} \mathcal{L} &= \nabla_{x_2} \mathcal{L} \odot H(x_1) & \nabla_{x_2} \mathcal{L} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} W_2 & \frac{\partial \mathcal{L}}{\partial \hat{y}} \\
 \nabla_{W_2} \mathcal{L} &= x_2^T \cdot \frac{\partial \mathcal{L}}{\partial \hat{y}}
 \end{aligned}$$

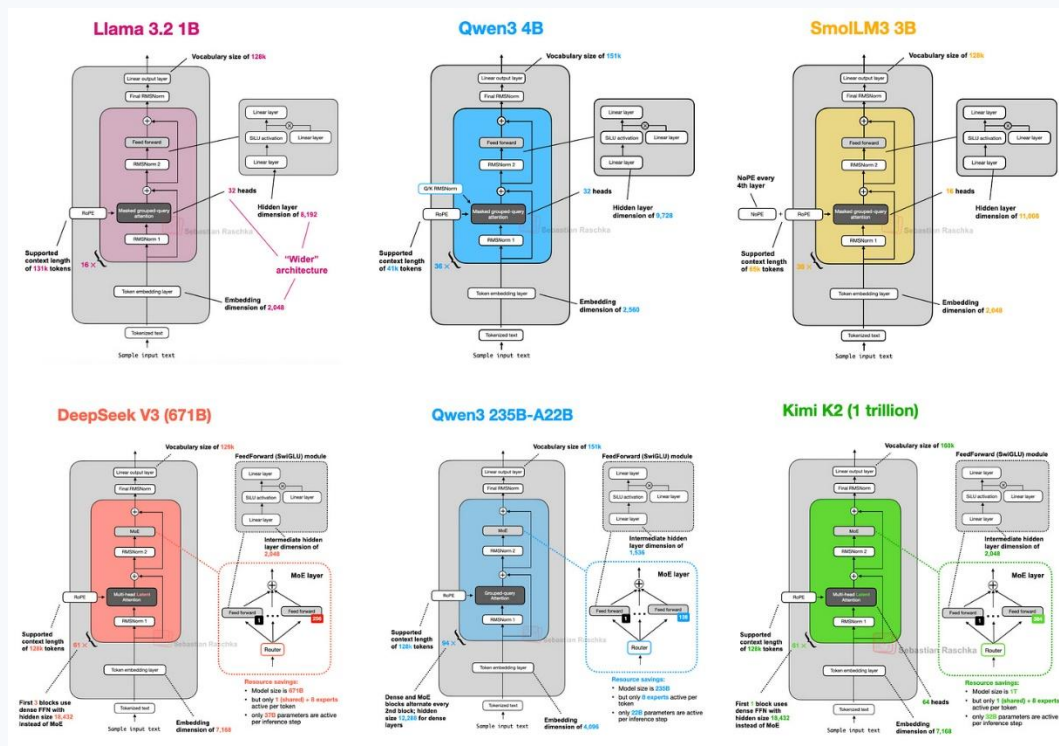
A neural network is

A computational graph
which can be trained end-to-end
with the help of error backpropagation



A neural network is

A computational graph
which can be trained end-to-end
with the help of error backpropagation

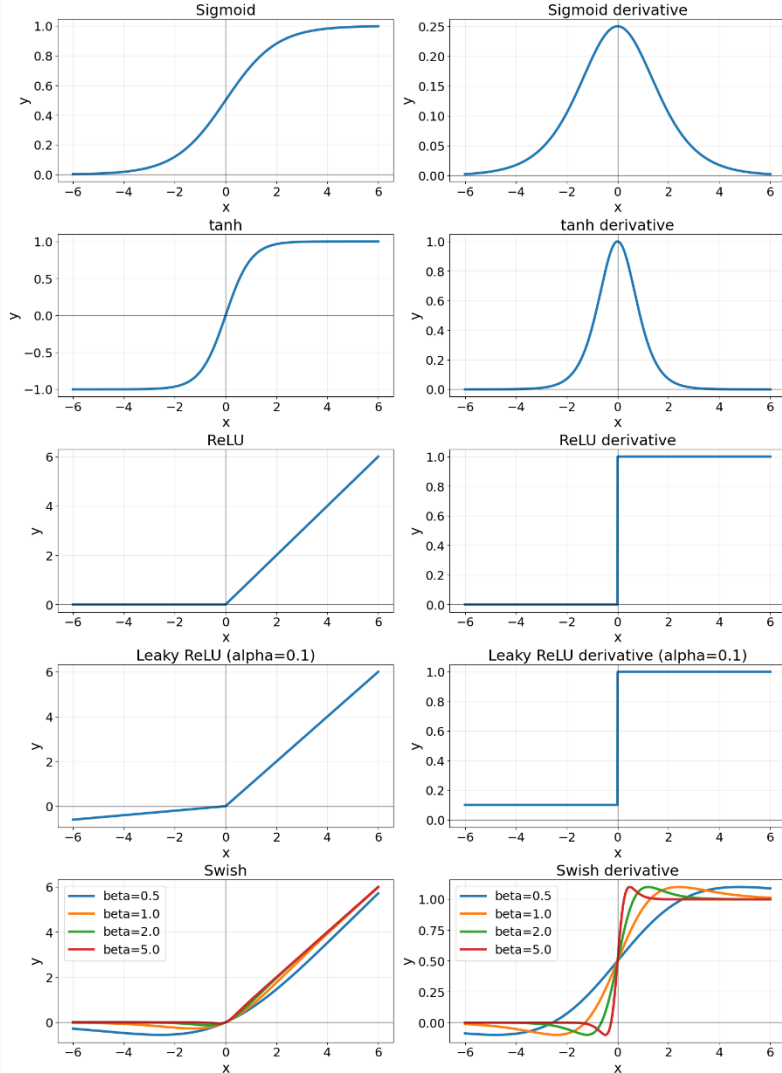


Some architectural
details and adaptations

Activation functions (nonlinearity)

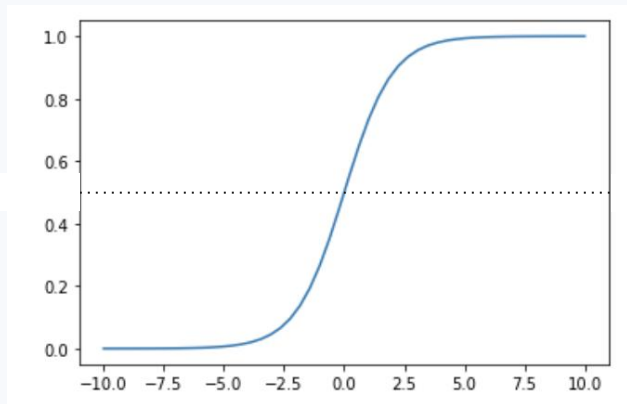
Several popular activations:

- Sigmoid: $\sigma(x) = \frac{1}{1+\exp(-x)}$
- Hyperbolic tangent: $\tanh(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$
- Rectified Linear Unit: $\text{ReLU}(x) = \max(0, x)$
- Leaky ReLU: $\begin{cases} x, & x > 0 \\ \alpha x, & x < 0 \end{cases}$
- Swish: $x \cdot \sigma(\beta x)$



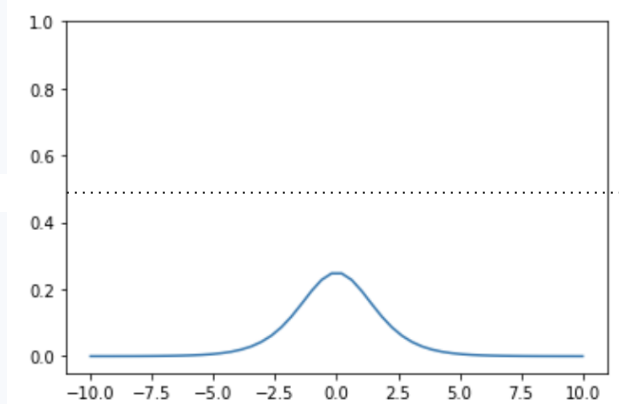
Sigmoid

Sigmoid function



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Derivative of Sigmoid



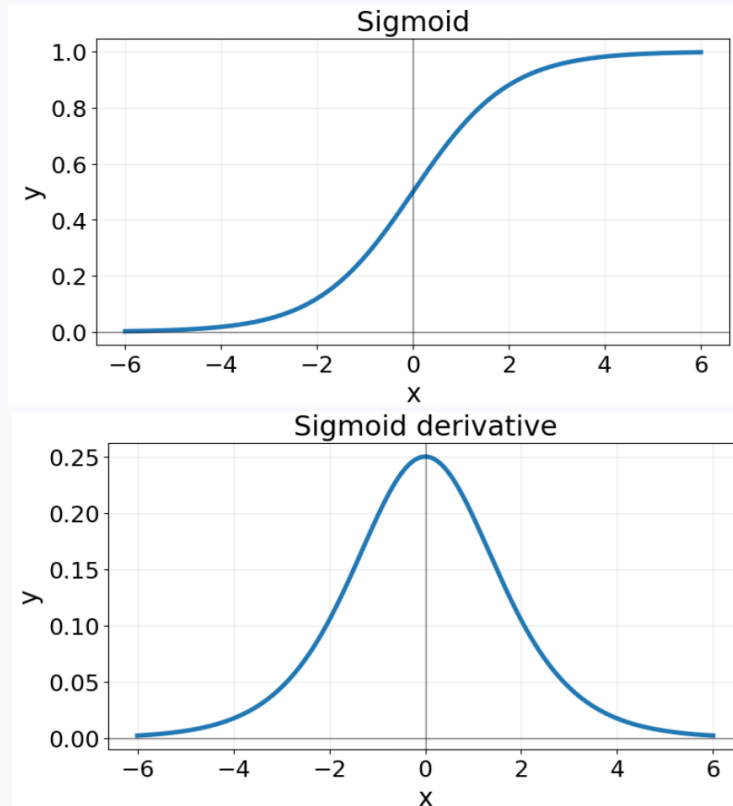
$$\sigma'(x) = \frac{e^{-x}}{(1 + e^{-x})^2}$$

Sigmoid – the saturation problem

$\hat{x}_i$
Sigmoid derivatives

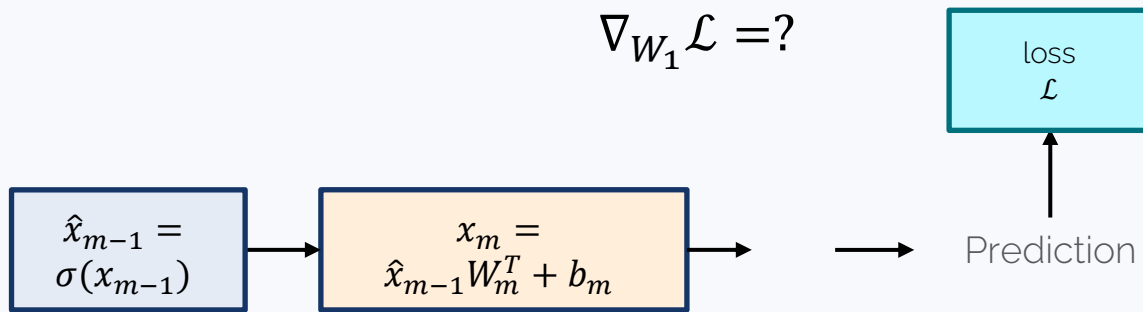
$$x_i = \sigma(x_{i-1}W_i + b_i)$$
$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial x_m} \frac{\partial x_m}{\partial \hat{x}_m} \frac{\partial \hat{x}_m}{\partial x_{m-1}} \frac{\partial x_{m-1}}{\partial \hat{x}_{m-1}} \cdots \frac{\partial x_1}{\partial \hat{x}_1} \frac{\partial \hat{x}_1}{\partial X}$$

1. $\sigma'(x) \leq \frac{1}{4}$. With many layers you'll get **vanishing gradients**
2. If you find yourself far from origin, $\sigma'(x) \approx 0$ and learning stops. This is known as **saturation of a sigmoid**



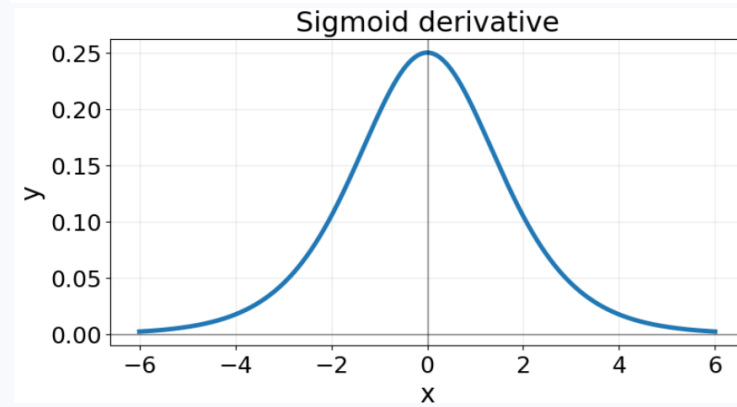
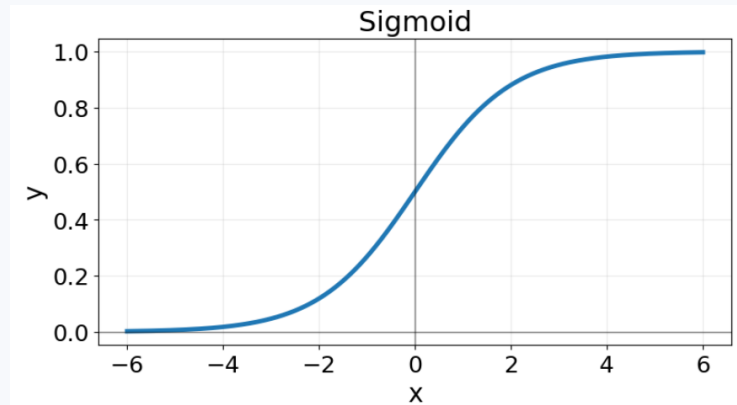
Sigmoid – the positivity problem

$$\nabla_{W_1} \mathcal{L} = ?$$



$$\nabla_{W_m} \mathcal{L} = \hat{x}_{m-1}^T \cdot \nabla_{x_m} \mathcal{L}$$

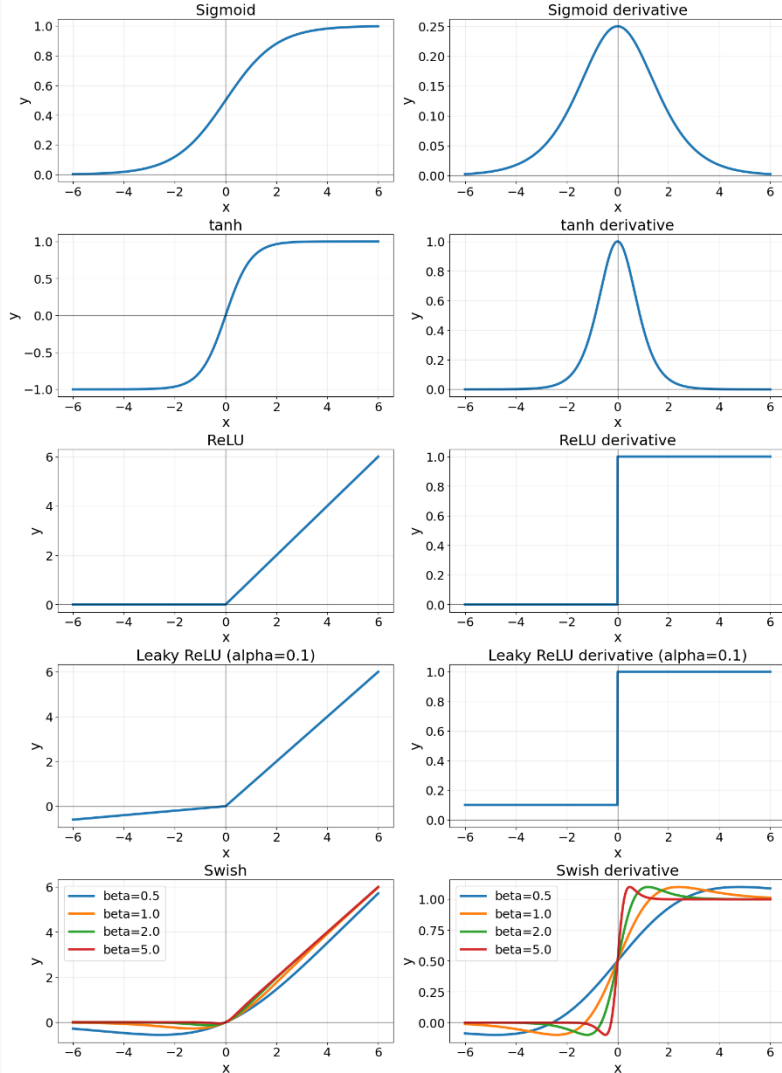
This will
always be
positive



Activation functions (nonlinearity)

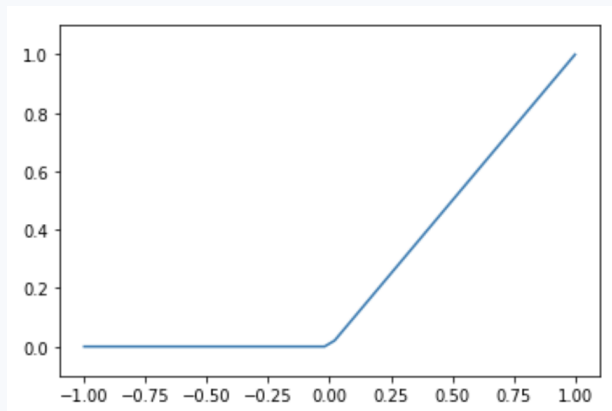
Several popular activations:

- Sigmoid: $\sigma(x) = \frac{1}{1+\exp(-x)}$
- Hyperbolic tangent: $\tanh(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$
- Rectified Linear Unit: $\text{ReLU}(x) = \max(0, x)$
- Leaky ReLU: $\begin{cases} x, & x > 0 \\ \alpha x, & x < 0 \end{cases}$
- Swish: $x \cdot \sigma(\beta x)$



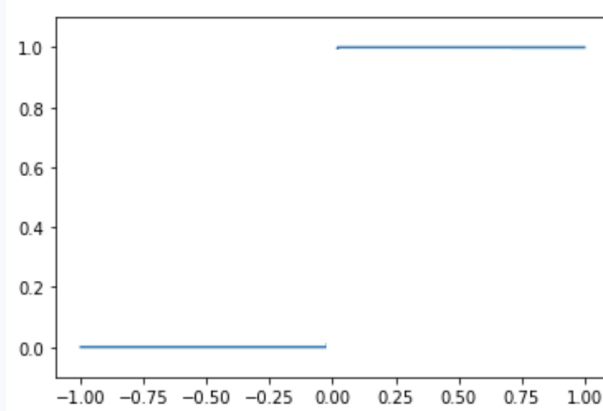
ReLU (Rectified Linear Unit)

ReLU function



$$\text{ReLU}(x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases}$$

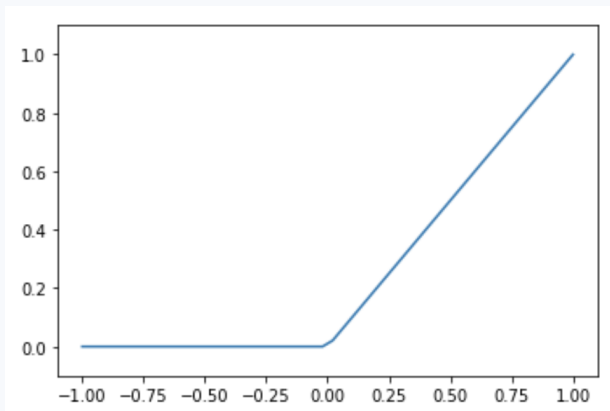
Derivative of ReLU



$$\text{ReLU}'(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases}$$

ReLU Problems

ReLU function



$$\text{ReLU}(x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases}$$



Outputs of ReLU are always non-negative, so we have the same problem as in Sigmoid

ReLU – the exploding gradients problem

$\hat{x}_i$

$x_i = \sigma(x_{i-1}W_i + b_i)$

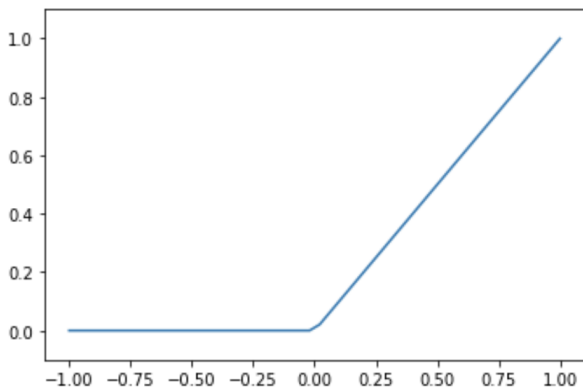
ReLU derivatives

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial x_m} \boxed{\frac{\partial x_m}{\partial \hat{x}_m}} \frac{\partial \hat{x}_m}{\partial x_{m-1}} \boxed{\frac{\partial x_{m-1}}{\partial \hat{x}_{m-1}}} \cdots \boxed{\frac{\partial x_1}{\partial \hat{x}_1}} \frac{\partial \hat{x}_1}{\partial X}$$

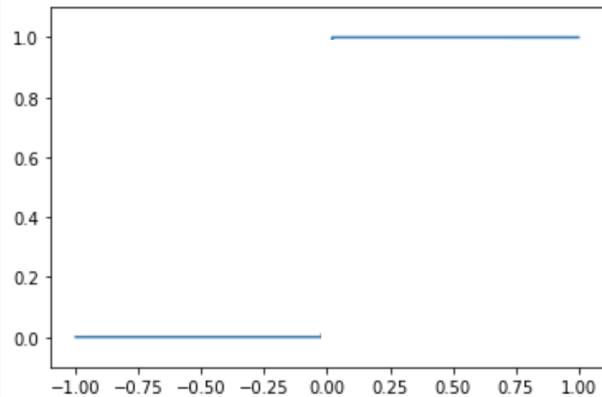
With too many layers you'll get **exploding gradients**

That's why they used `tanh` in RNNs (as many layers as there are tokens in a text)

ReLU function



Derivative of ReLU



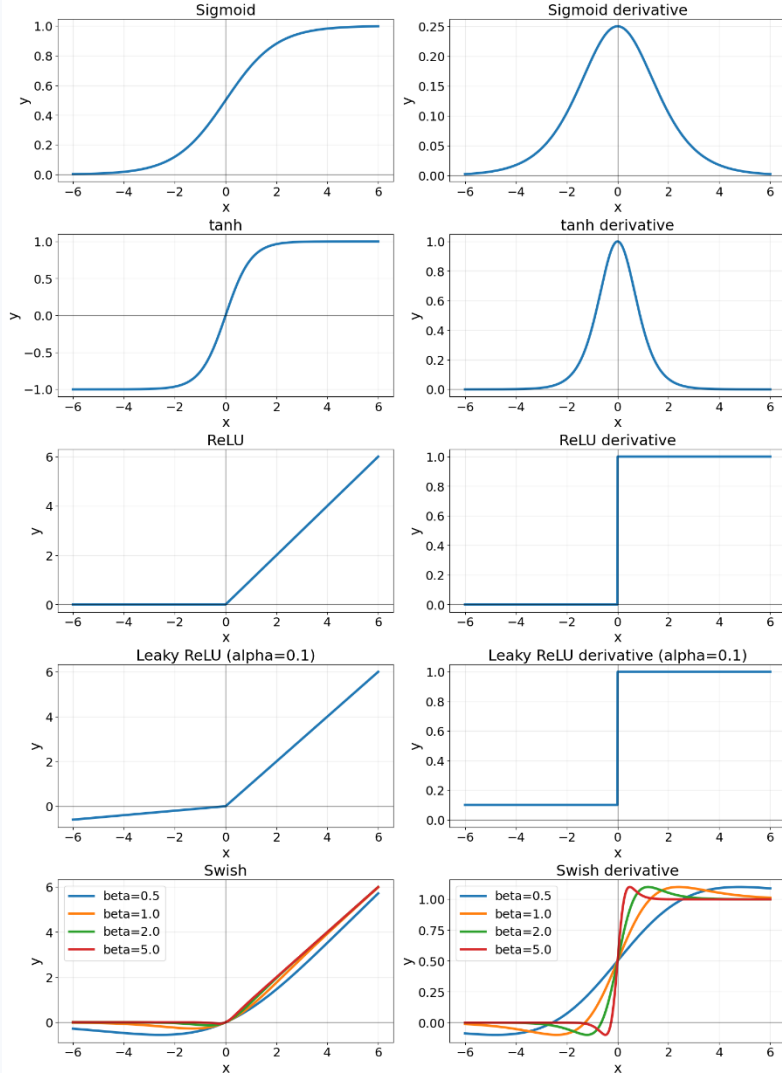
Takeaways

- **Sigmoid** is only used in special locations: binary classification finale and gating mechanisms (where you really need numbers between 0 and 1)
- **Tanh** also has the saturation issue. It was used in RNNs (very deep networks; helped to avoid gradient explosion)
- **ReLU** helped CV a lot, but it's always better to have something smooth. That's why there were many attempts at creating surrogates that are both smooth and not painful to compute (such as **swish**)

Activation functions (nonlinearity)

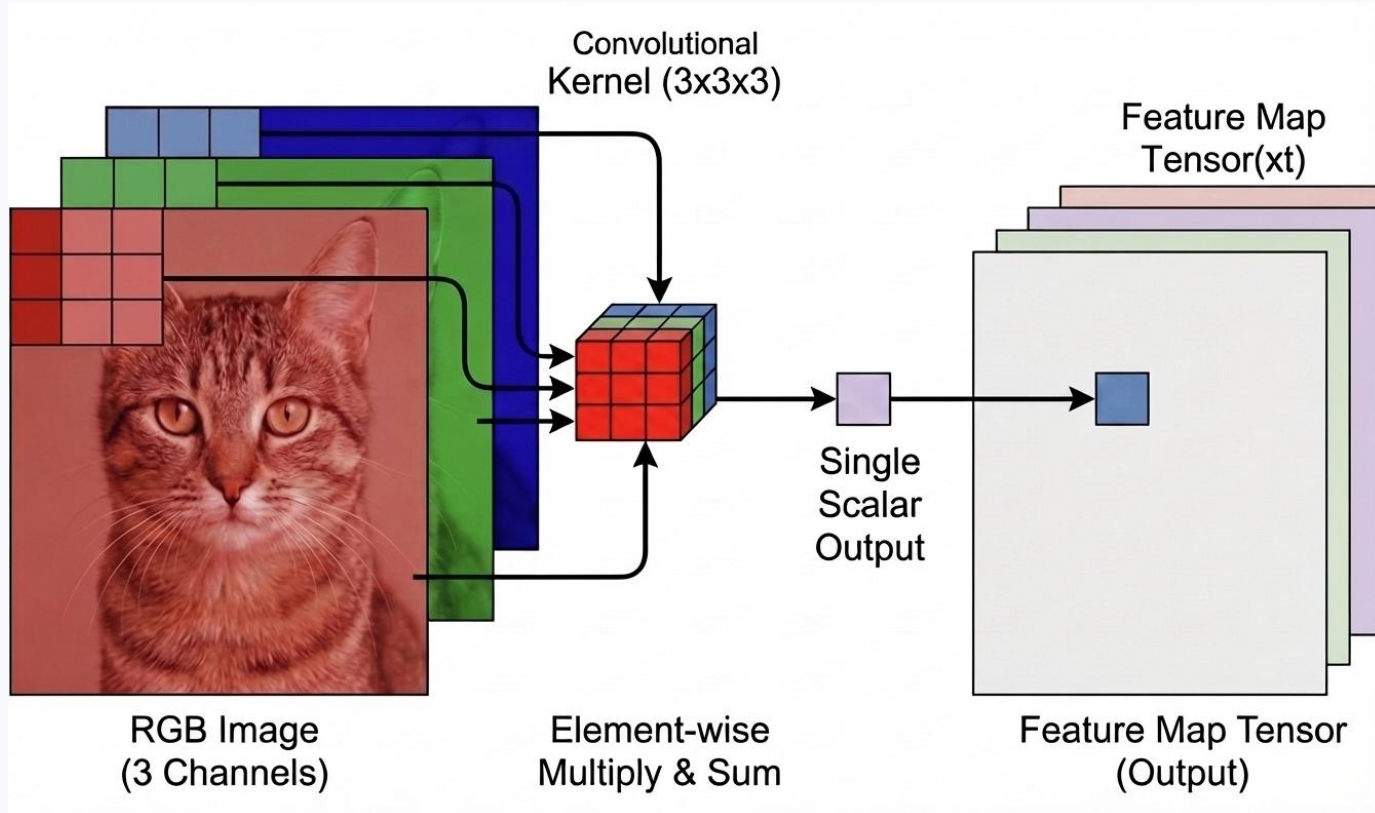
Several popular activations:

- Sigmoid: $\sigma(x) = \frac{1}{1+\exp(-x)}$
- Hyperbolic tangent: $\tanh(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$
- Rectified Linear Unit: $\text{ReLU}(x) = \max(0, x)$
- Leaky ReLU: $\begin{cases} x, & x > 0 \\ \alpha x, & x < 0 \end{cases}$
- Swish: $x \cdot \sigma(\beta x)$



Architectural adaptations

Example: Convolutional Networks (CNNs)



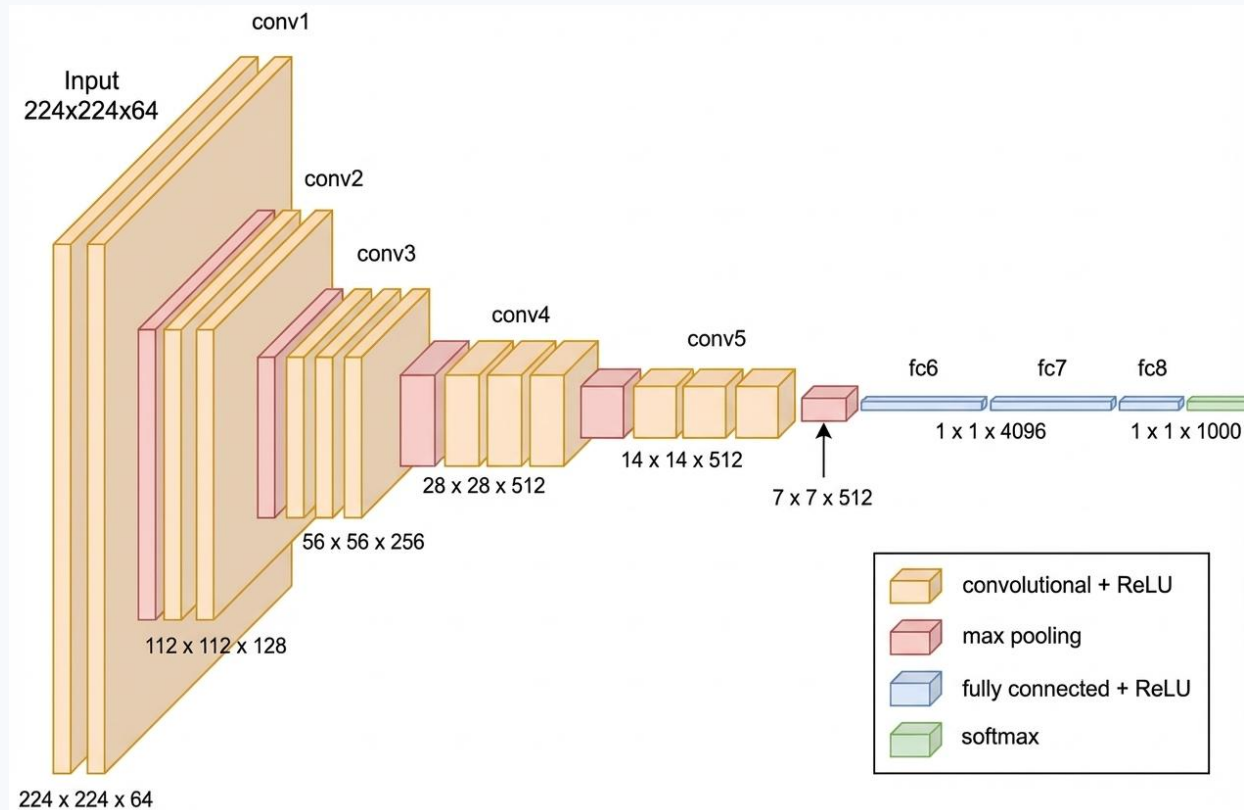
Convolutional architectures

Max pooling:

12	5	2	3
9	1	8	6
4	10	2	7
5	8	3	9



12	8
10	9



2012 – AlexNet starts DL revolution in CV

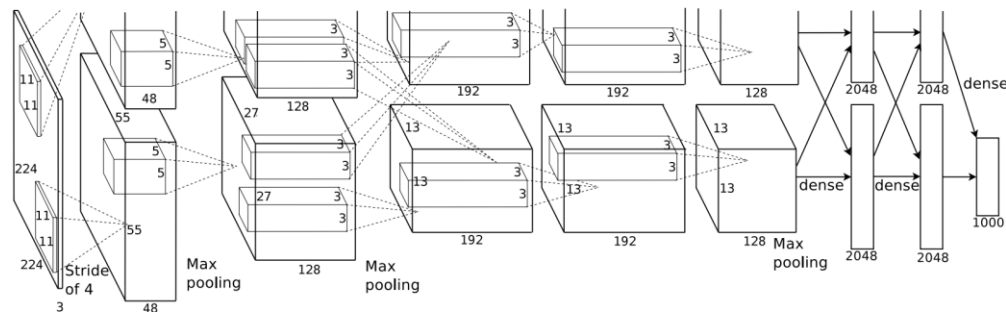
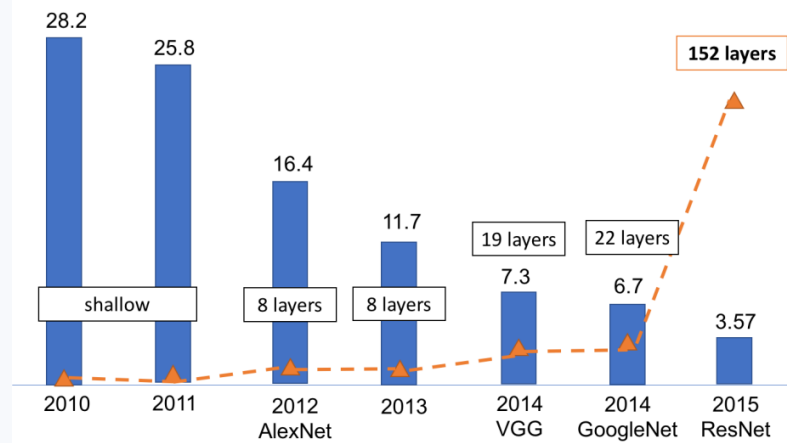


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

ImageNet Classification with Deep Convolutional
Neural Networks

Krizhevsky, Sutskever, and Hinton

https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

Residual connections

- Allow gradients to flow straight to the deepest layers
- Enabled training of very deep networks

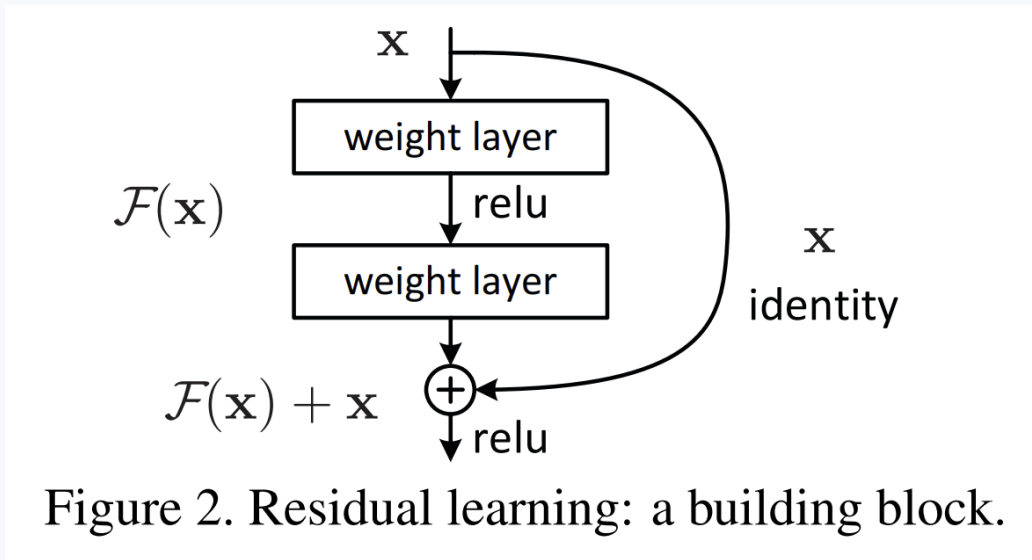
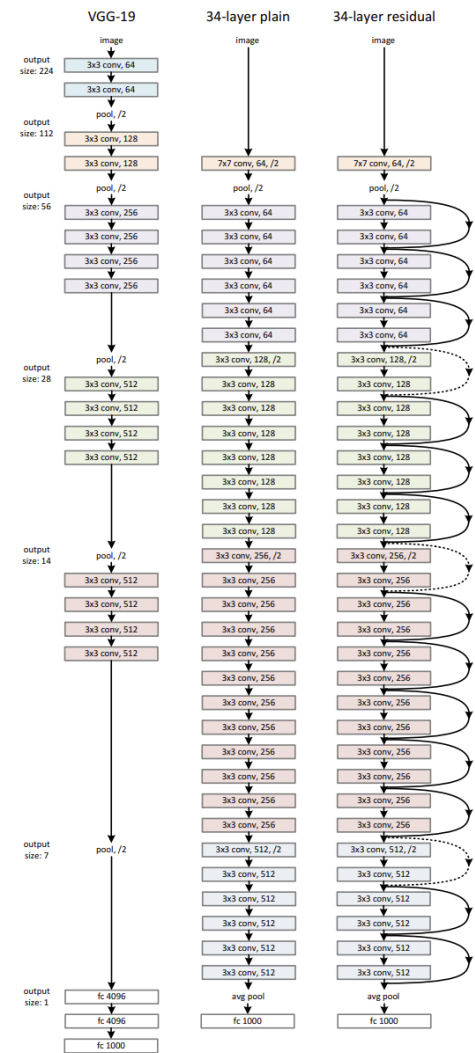


Figure 2. Residual learning: a building block.



Regularization

What is regularization?

- Countering overfitting
- Stabilizing optimization
- Imposing conditions

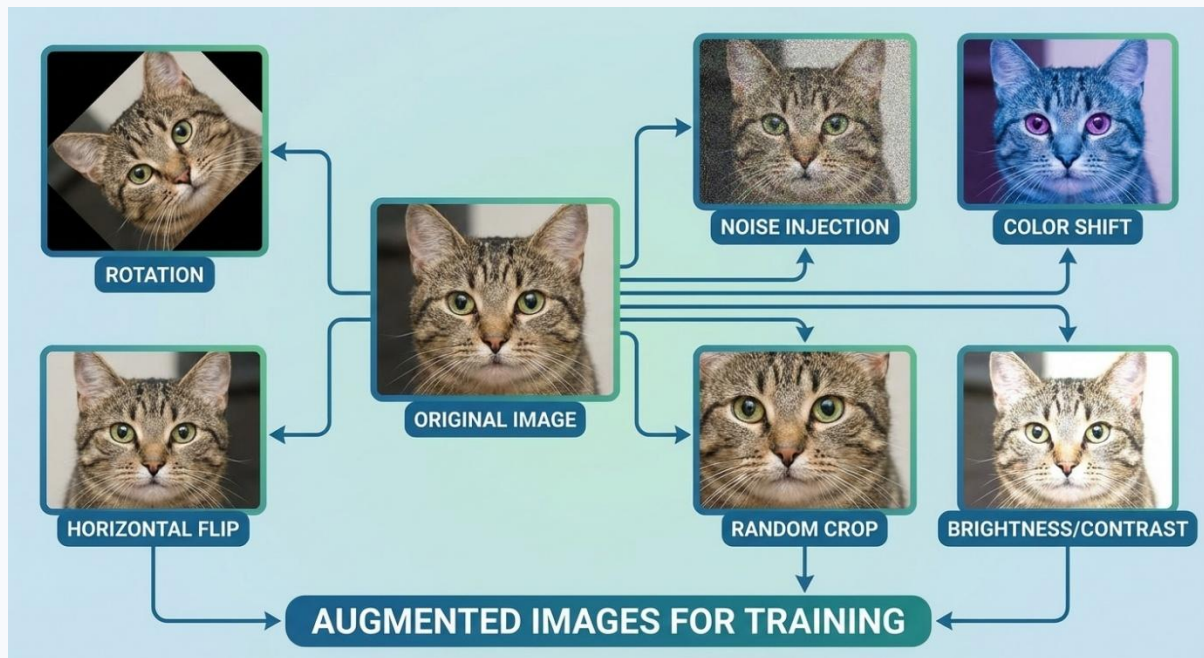
Data augmentation

Classic recipe for images:

- Rotations
- Blur
- Colour correction
- ...

Modern recipe:

- Generated data 🤖



Dropout

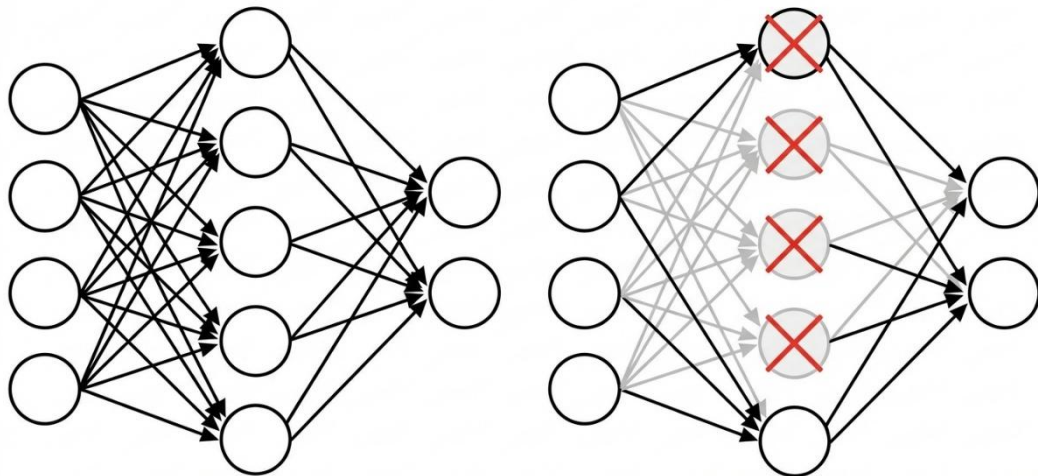
During training, insert:

$$x_{m+1} = x_m \odot \text{mask},$$

where $\text{mask} \sim \text{Bernoulli}(1 - p)$

During inference: ignore this

Dropout in Neural Networks



Dropout

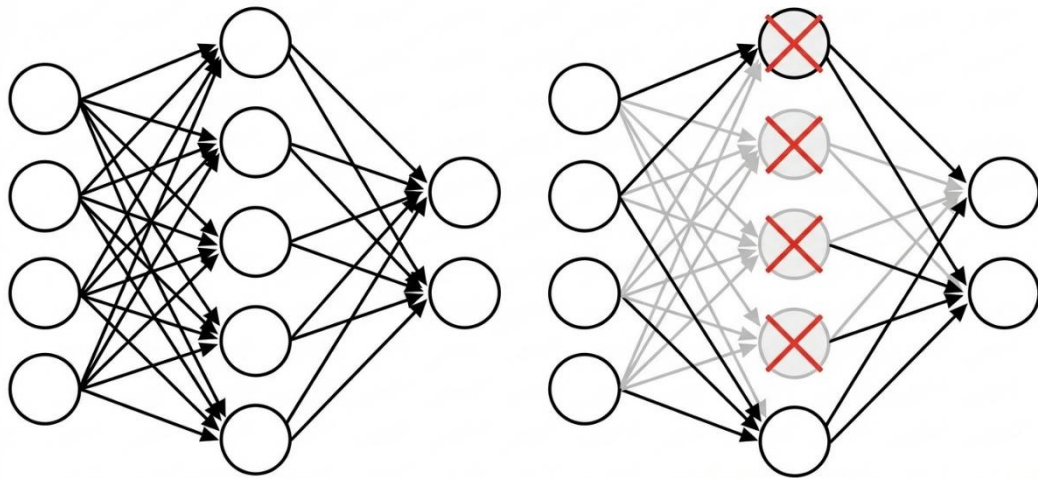
During training, insert:

$$x_{m+1} = \frac{1}{1-p} x_m \odot \text{mask},$$

where $\text{mask} \sim \text{Bernoulli}(1-p)$

$$\nabla_{x_m} \mathcal{L} = \frac{1}{1-p} \nabla_{x_{m+1}} \mathcal{L} \odot \text{mask}$$

Dropout in Neural Networks



BatchNorm

During training:

$$x_{k+1} = \frac{x_k - \mu(\text{batch})}{\sqrt{\sigma^2(\text{batch}) + \varepsilon}},$$

Then

$$x_{k+2} = \beta x_{k+1} + \gamma,$$

where β and γ are trainable

During inference:

$$\mu_* = \mu_* \lambda + \mu(\text{batch}) \cdot (1 - \lambda),$$

$$\sigma_*^2 = \sigma_*^2 \lambda + \sigma^2(\text{batch}) \cdot (1 - \lambda),$$

Why does it work?

Original explanation:

Gradient descent changes weights, which changes the distribution of incoming data to the next layers.

During inference:

$$\mu_* = \mu_* \lambda + \mu(\text{batch}) \cdot (1 - \lambda),$$

$$\sigma_*^2 = \sigma_*^2 \lambda + \sigma^2(\text{batch}) \cdot (1 - \lambda),$$

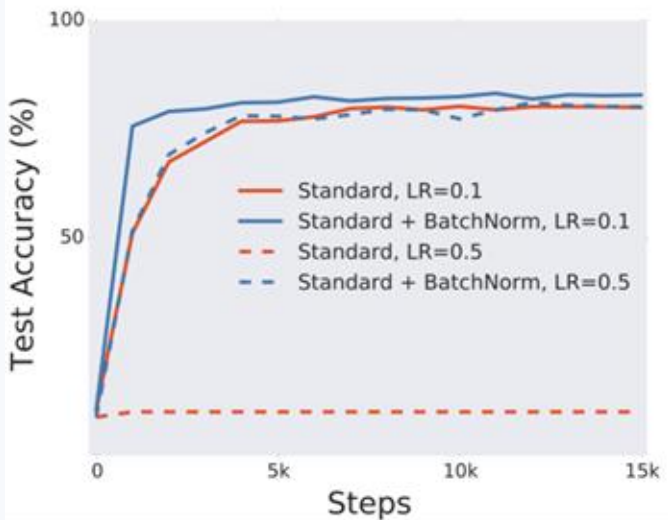
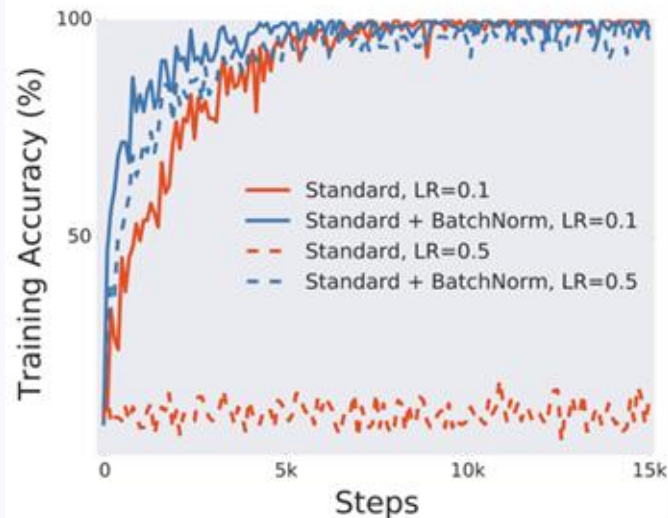
Why does it work?

Original explanation:

Gradient descent changes weights, which changes the distribution of incoming data to the next layers.

How Does Batch Normalization Help Optimization?

<https://arxiv.org/pdf/1805.11604>



Why does it work?

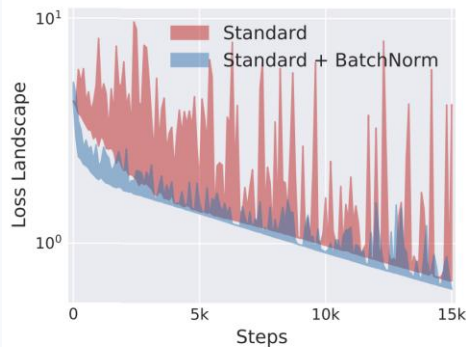
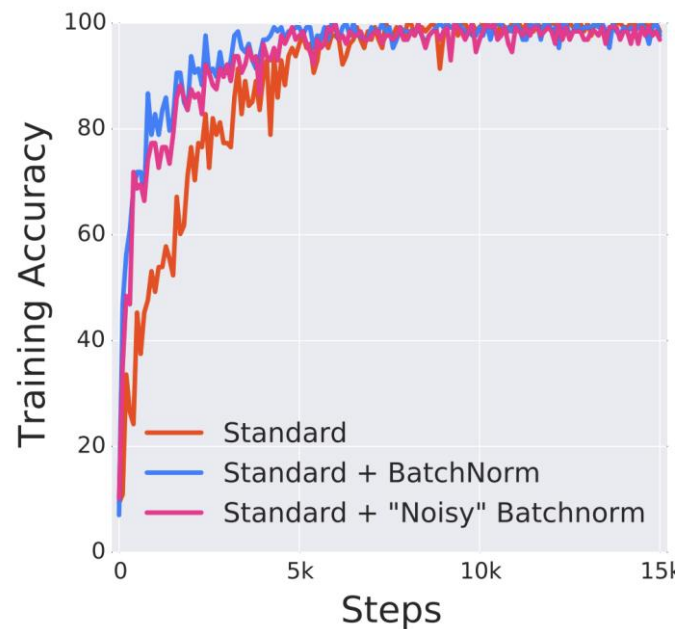
Original explanation:

Gradient descent changes weights, which changes the distribution of incoming data to the next layers.

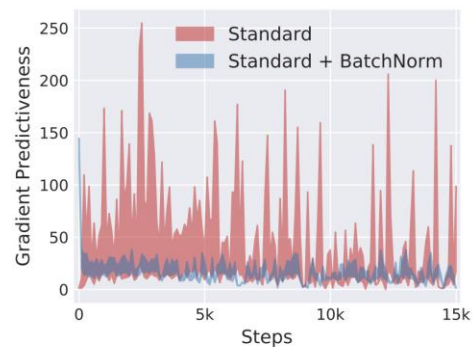
Critique:

Varying distribution doesn't hurt that much!

It might be smoothing things though



(a) loss landscape



(b) gradient predictiveness

How Does Batch Normalization Help Optimization?

<https://arxiv.org/pdf/1805.11604>

Why does it work?

Theorem 4.1 (The effect of BatchNorm on the Lipschitzness of the loss). *For a BatchNorm network with loss $\hat{\mathcal{L}}$ and an identical non-BN network with (identical) loss $\mathcal{L}$,*

$$\left\| \nabla_{\mathbf{y}_j} \hat{\mathcal{L}} \right\|^2 \leq \frac{\gamma^2}{\sigma_j^2} \left(\left\| \nabla_{\mathbf{y}_j} \mathcal{L} \right\|^2 - \frac{1}{m} \langle \mathbf{1}, \nabla_{\mathbf{y}_j} \mathcal{L} \rangle^2 - \frac{1}{m} \langle \nabla_{\mathbf{y}_j} \mathcal{L}, \hat{\mathbf{y}}_j \rangle^2 \right).$$

Theorem 4.2 (The effect of BN to smoothness). *Let $\hat{\mathbf{g}}_j = \nabla_{\mathbf{y}_j} \mathcal{L}$ and $\mathbf{H}_{jj} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}_j \partial \mathbf{y}_j}$ be the gradient and Hessian of the loss with respect to the layer outputs respectively. Then*

$$\left(\nabla_{\mathbf{y}_j} \hat{\mathcal{L}} \right)^\top \frac{\partial \hat{\mathcal{L}}}{\partial \mathbf{y}_j \partial \mathbf{y}_j} \left(\nabla_{\mathbf{y}_j} \hat{\mathcal{L}} \right) \leq \frac{\gamma^2}{\sigma^2} \left(\frac{\partial \hat{\mathcal{L}}}{\partial \mathbf{y}_j} \right)^\top \mathbf{H}_{jj} \left(\frac{\partial \hat{\mathcal{L}}}{\partial \mathbf{y}_j} \right) - \frac{\gamma}{m\sigma^2} \langle \hat{\mathbf{g}}_j, \hat{\mathbf{y}}_j \rangle \left\| \frac{\partial \hat{\mathcal{L}}}{\partial \mathbf{y}_j} \right\|^2$$

If we also have that the $\mathbf{H}_{jj}$ preserves the relative norms of $\hat{\mathbf{g}}_j$ and $\nabla_{\mathbf{y}_j} \hat{\mathcal{L}}$,

$$\left(\nabla_{\mathbf{y}_j} \hat{\mathcal{L}} \right)^\top \frac{\partial \hat{\mathcal{L}}}{\partial \mathbf{y}_j \partial \mathbf{y}_j} \left(\nabla_{\mathbf{y}_j} \hat{\mathcal{L}} \right) \leq \frac{\gamma^2}{\sigma^2} \left(\hat{\mathbf{g}}_j^\top \mathbf{H}_{jj} \hat{\mathbf{g}}_j - \frac{1}{m\gamma} \langle \hat{\mathbf{g}}_j, \hat{\mathbf{y}}_j \rangle \left\| \frac{\partial \hat{\mathcal{L}}}{\partial \mathbf{y}_j} \right\|^2 \right)$$

Initialization

Main idea: variance shouldn't change across the layers

Example: **Kaiming initialization** for networks with **ReLU**:

$$x = \text{ReLU}(x_{prev}); \quad \text{let } \mathbb{E}(x_{prev}) = 0$$

$$\begin{aligned}\mathbb{V}(xw^T) &= \sum_i \mathbb{E}(x_i)^2 \mathbb{V}(w_i) + \mathbb{V}(x_i) \cancel{\mathbb{E}(w_i)^2} + \mathbb{V}(x_i) \mathbb{V}(w_i) = \\ &= n_{in} \mathbb{V}(w_i) \mathbb{E}(x_i^2) = \frac{1}{2} n_{in} \mathbb{V}(w_i) \mathbb{V}(x_{prev})\end{aligned}$$

Take: $w_i \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$.

About the superior
power of neural
networks

The universal approximation theorem (Cybenko)

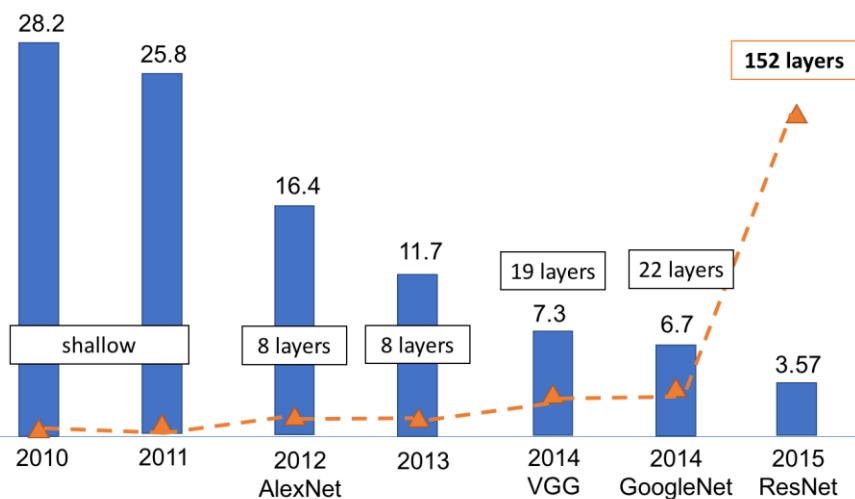
Universal approximation theorem—Let $C(X, \mathbb{R}^m)$ denote the set of **continuous functions** from a subset X of a Euclidean $\mathbb{R}^n$ space to a Euclidean space $\mathbb{R}^m$. Let $\sigma \in C(\mathbb{R}, \mathbb{R})$. Note that $(\sigma \circ x)_i = \sigma(x_i)$, so $\sigma \circ x$ denotes σ applied to each component of x .

Then σ is not **polynomial if and only if** for every $n \in \mathbb{N}$, $m \in \mathbb{N}$, **compact** $K \subseteq \mathbb{R}^n$, $f \in C(K, \mathbb{R}^m)$, $\varepsilon > 0$ there exist $k \in \mathbb{N}$, $A \in \mathbb{R}^{k \times n}$, $b \in \mathbb{R}^k$, $C \in \mathbb{R}^{m \times k}$ such that

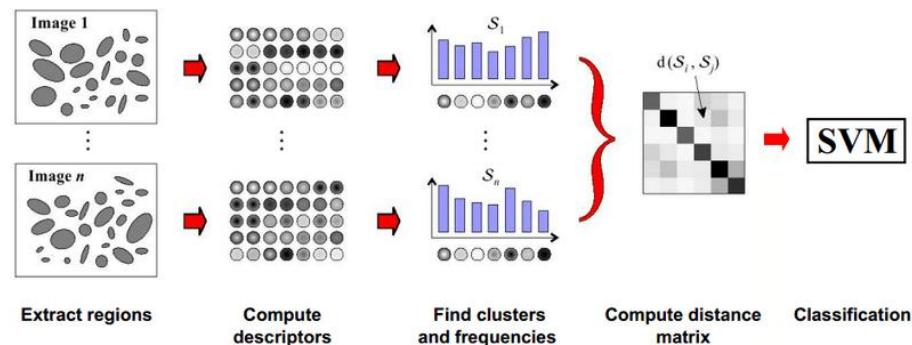
$$\sup_{x \in K} \|f(x) - g(x)\| < \varepsilon$$

where $g(x) = C \cdot (\sigma \circ (A \cdot x + b))$

The DL revolution of 2012



Before:



After:

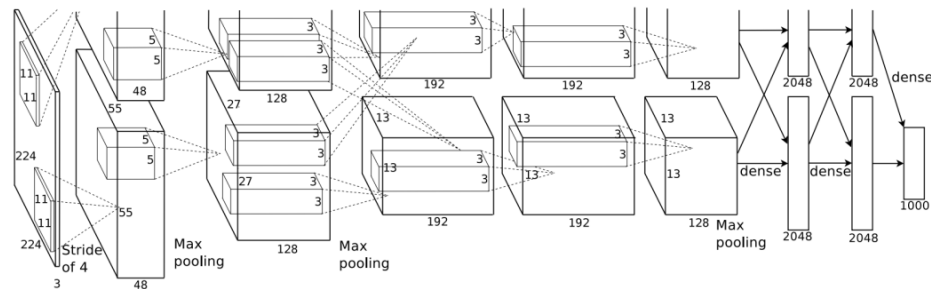
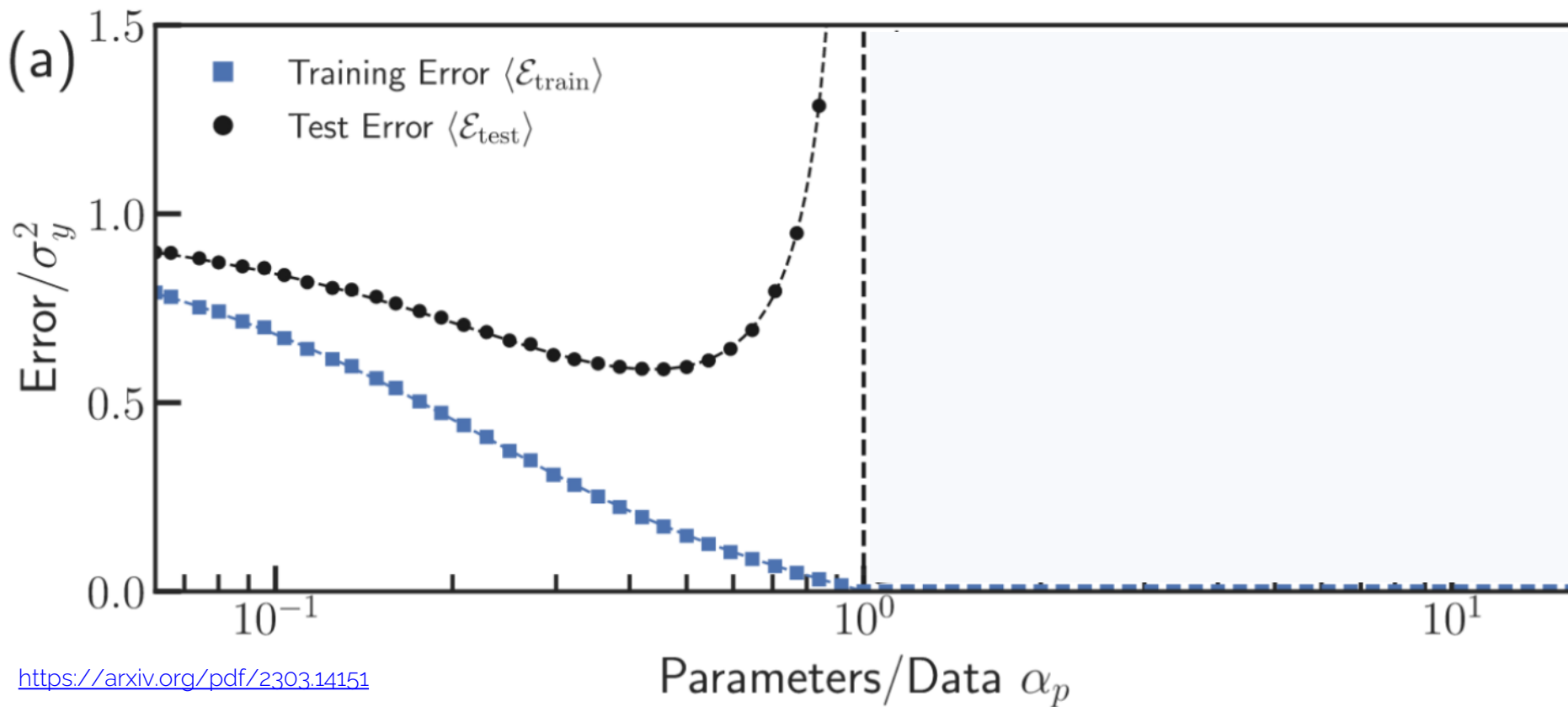


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

A classical view on complexity vs generalization

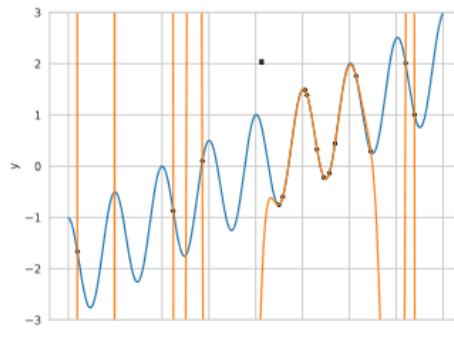
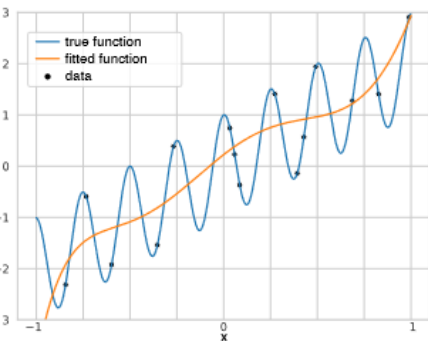


Approximation vs interpolation

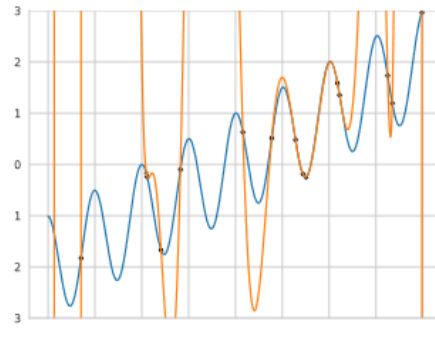
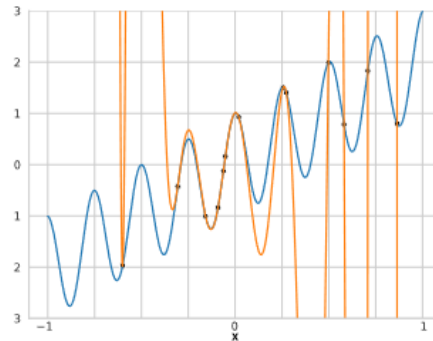
N points, D functions

$$\hat{w} = (X^T X)^{-1} X^T y$$

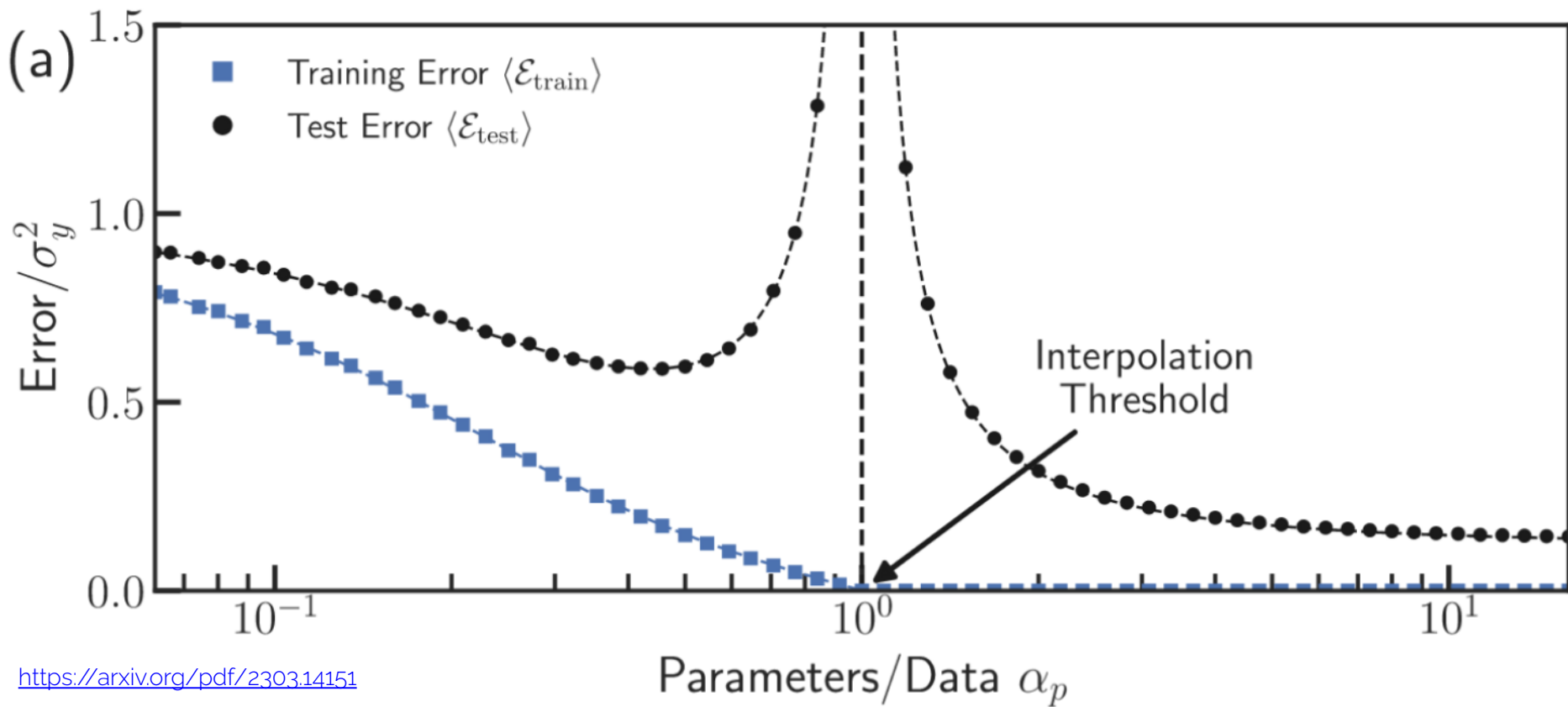
Interpolation (train error = 0) for $N = D$
underparametrized



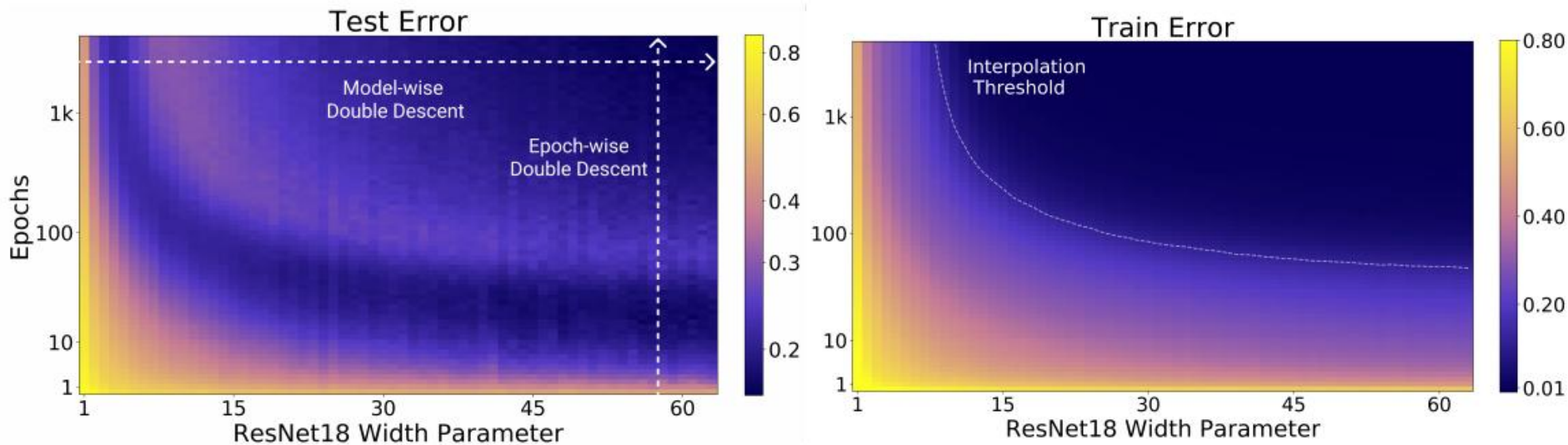
interpolation threshold



Double descent



Double descent



<https://www.lesswrong.com/posts/FRv7ryoqtvSuqBxuT/understanding-deep-double-descent>
<https://openai.com/index/deep-double-descent/>

When $D > N$, solve

Approximation vs interpolation

N points, D functions

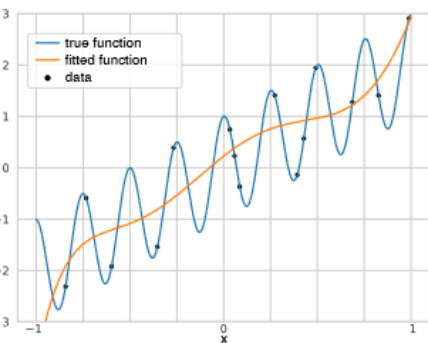
$$\begin{cases} \|w\| \rightarrow \min \\ Xw^T = y \end{cases}$$

$$\hat{w}^T = (X^T X)^{-1} X^T y$$

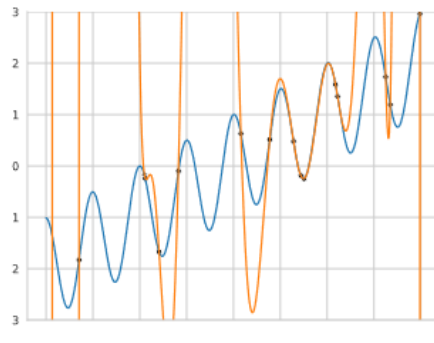
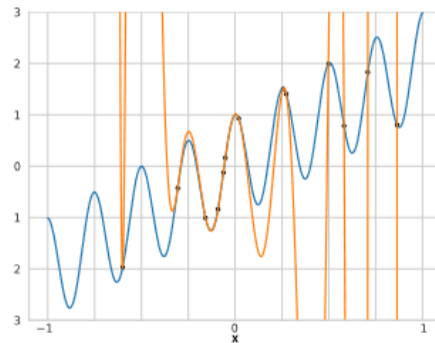
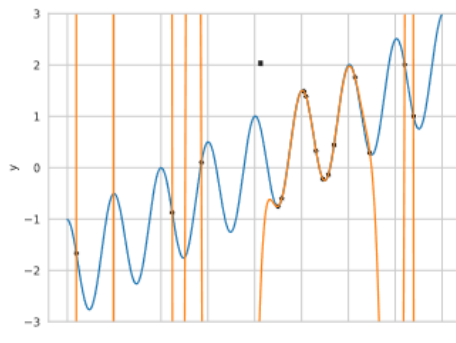
$$\hat{w}^T = X^T (X X^T)^{-1} y$$

Interpolation (train error = 0) for $N = D$

underparametrized



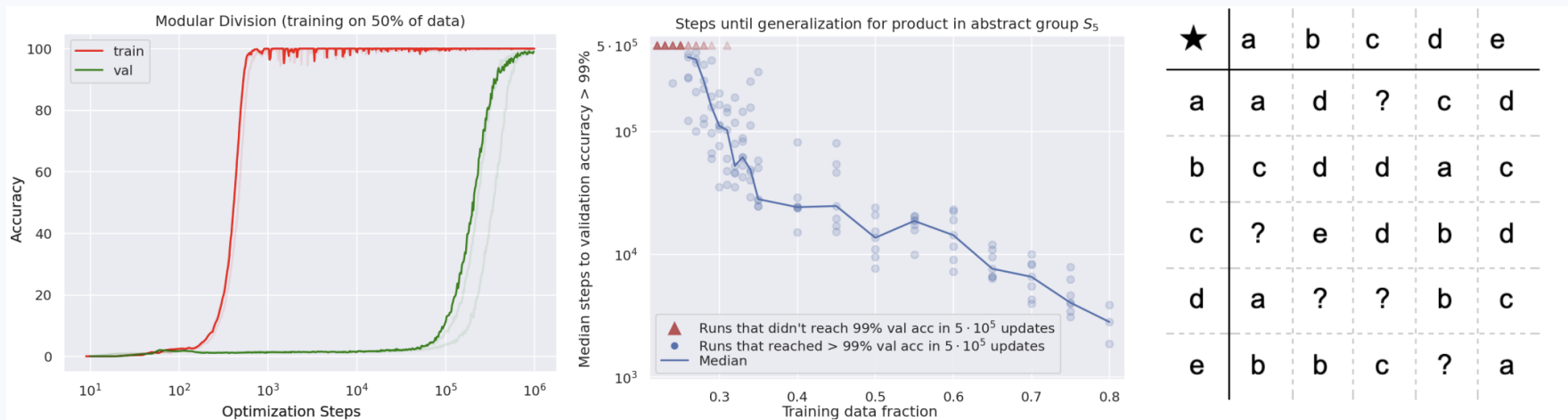
interpolation threshold



Grokking

Grokking: generalization beyond overfitting on small algorithmic datasets

<https://arxiv.org/pdf/2201.02177>



An attempt at explaining grokking

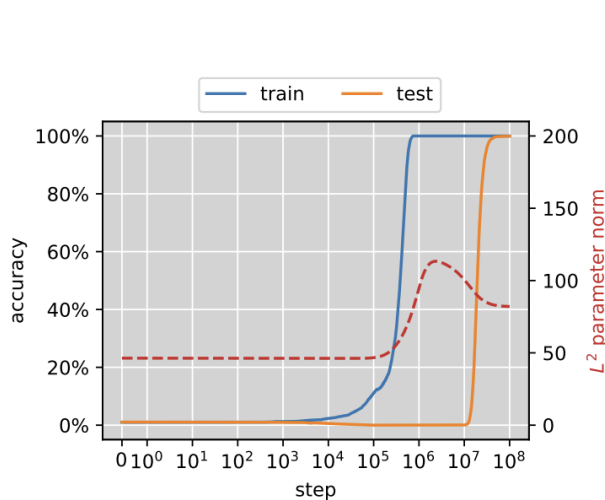
Dichotomy of Early and Late
Phase Implicit Biases Can
Provably Induce Grokking

<https://arxiv.org/pdf/2311.1881>

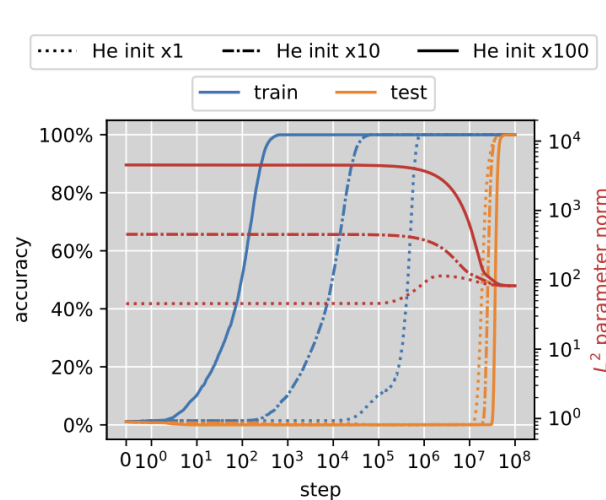
7

Two training phases for an overparametrized model:

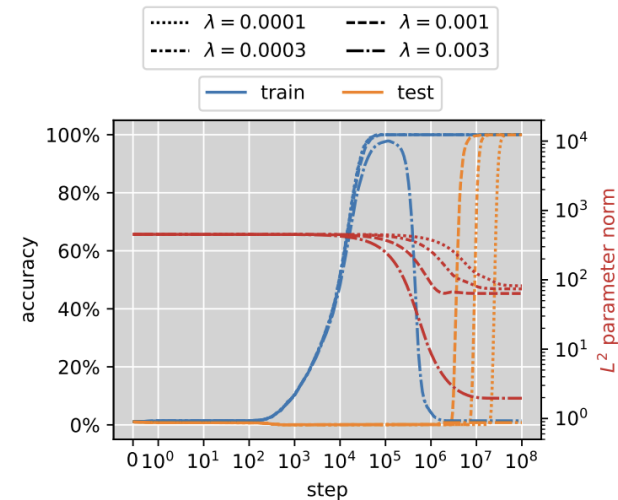
1. Keeping roughly the original direction of weights, the model is able to memorize the training data
2. Weight decay (L2 regularization) wins over, and training starts moving towards the min-norm solution



(a) He init, $\lambda = 10^{-4}$



(b) Effect of Initialization Scale



(c) Effect of Weight Decay

Why NNs aren't the magic pill

- Not very good for heterogeneous data
- Not interpretable; might give a function which is ridiculous from theoretical point of view
- Require lots of data; often finicky to train
- Only work with measure > 0 phenomena

LIME

Tries to explain the prediction for a particular object

- Take several closest data points
- Train a simple model (linear) on model's predictions for those objects
- Look at the coefficients

Machine learning Fano varieties and terminal singularities

The two papers

- **Machine learning the dimension of a Fano variety** (Nature)
<https://www.nature.com/articles/s41467-023-41157-1>
<https://arxiv.org/pdf/2309.05473>
- **Machine learning detects terminal singularities** (NeurIPS '23)
<https://arxiv.org/pdf/2310.20458>

Weighted projective spaces

Weighted projective space: $\mathbb{P}(a_1, \dots, a_n) := (\mathbb{A}^{n+1} \setminus \{0\}) / \mathbb{C}^\times$,
 $t: (x_0, \dots, x_n) \mapsto (t^{a_0}x_0, \dots, t^{a_n}x_n)$

Weighted projective spaces

Weighted projective space: $\mathbb{P}(a_1, \dots, a_n) := (\mathbb{A}^{n+1} \setminus \{0\}) / \mathbb{C}^\times$,
 $t: (x_0, \dots, x_n) \mapsto (t^{a_0}x_0, \dots, t^{a_n}x_n)$

Example 1: $\mathbb{P}(1,1,2)$; the action is: $(x_0, x_1, x_2) \mapsto (tx_0, tx_1, t^2x_2)$.

Describing orbits: if $x_2 \neq 0$, it contains $(?, ?, 1)$. Stabilizer: $t^2 = 1$.

Let's map $\mathbb{A}^3 \setminus \{x_2 = 0\} \rightarrow \mathbb{A}^2$, $(x_0, x_1, x_2) \mapsto (x_0, x_1)$

Residue action of $\mathbb{Z}_2$: $(x_0, x_1) \mapsto (-x_0, -x_1)$.

Invariants: $\mathbb{C}[\mathbb{A}^2]^{\mathbb{Z}_2} = \mathbb{C}[x_0^2, x_0x_1, x_1^2] \hookrightarrow \mathbb{C}[x_0, x_1]$. This defines $\mathbb{A}^2/\mathbb{C}^\times \leftarrow \mathbb{A}^2$:

$(x_0, x_1) \mapsto (x_0^2, x_0x_1, x_1^2) = (X, Y, Z)$. The image is the cone $XZ = Y^2$.

Weighted projective spaces

Weighted projective space: $\mathbb{P}(a_1, \dots, a_n) := (\mathbb{A}^{n+1} \setminus \{0\}) / \mathbb{C}^\times$,
 $t: (x_0, \dots, x_n) \mapsto (t^{a_0}x_0, \dots, t^{a_n}x_n)$

Example 1: $\mathbb{P}(1,1,2)$; the action is: $(x_0, x_1, x_2) \mapsto (tx_0, tx_1, t^2x_2)$.

Describing orbits: if $x_2 \neq 0$, it contains $(?, ?, 1)$. Stabilizer: $t^2 = 1$.

Weighted projective spaces

Weighted projective space: $\mathbb{P}(a_1, \dots, a_n) := (\mathbb{A}^{n+1} \setminus \{0\}) / \mathbb{C}^\times$,
 $t: (x_0, \dots, x_n) \mapsto (t^{a_0}x_0, \dots, t^{a_n}x_n)$

Example 1: $\mathbb{P}(1,1,2)$; the action is: $(x_0, x_1, x_2) \mapsto (tx_0, tx_1, t^2x_2)$.

Describing orbits: if $x_2 \neq 0$, it contains $(?, ?, 1)$. Stabilizer: $t^2 = 1$.

Let's map $\mathbb{A}^3 \setminus \{x_2 = 0\} \rightarrow \mathbb{A}^2$, $(x_0, x_1, x_2) \mapsto (x_0, x_1)$

Residue action of $\mathbb{Z}_2$: $(x_0, x_1) \mapsto (-x_0, -x_1)$.

Weighted projective spaces

Weighted projective space: $\mathbb{P}(a_1, \dots, a_n) := (\mathbb{A}^{n+1} \setminus \{0\}) / \mathbb{C}^\times$,
 $t: (x_0, \dots, x_n) \mapsto (t^{a_0}x_0, \dots, t^{a_n}x_n)$

Example 1: $\mathbb{P}(1,1,2)$; the action is: $(x_0, x_1, x_2) \mapsto (tx_0, tx_1, t^2x_2)$.

In the chart $x_2 \neq 0$ it's a cone $XZ = Y^2$; $(x_0, x_1) \mapsto (x_0^2, x_0x_1, x_1^2) = (X, Y, Z)$

Weighted projective spaces

Weighted projective space: $\mathbb{P}(a_1, \dots, a_n) := (\mathbb{A}^{n+1} \setminus \{0\}) / \mathbb{C}^\times$,
 $t: (x_0, \dots, x_n) \mapsto (t^{a_0}x_0, \dots, t^{a_n}x_n)$

Example 1: $\mathbb{P}(1,1,2)$; the action is: $(x_0, x_1, x_2) \mapsto (tx_0, tx_1, t^2x_2)$.

In the chart $x_2 \neq 0$ it's a cone $XZ = Y^2$; $(x_0, x_1) \mapsto (x_0^2, x_0x_1, x_1^2) = (X, Y, Z)$

Resolution of singularities: $(u, v) \mapsto (u, uv, uv^2)$. We get $(0,0,0)$ if $u = 0$ and v is arbitrary.

Weighted projective spaces

Weighted projective space: $\mathbb{P}(a_1, \dots, a_n) := (\mathbb{A}^{n+1} \setminus \{0\}) / \mathbb{C}^\times$,
 $t: (x_0, \dots, x_n) \mapsto (t^{a_0}x_0, \dots, t^{a_n}x_n)$

Example 1: $\mathbb{P}(1,1,2)$; the action is: $(x_0, x_1, x_2) \mapsto (tx_0, tx_1, t^2x_2)$.

In the chart $x_2 \neq 0$ it's a cone $XZ = Y^2$; $(x_0, x_1) \mapsto (x_0^2, x_0x_1, x_1^2) = (X, Y, Z)$

Resolution of singularities: $(u, v) \mapsto (u, uv, uv^2)$. We get $(0,0,0)$ if $u = 0$ and v is arbitrary.

On the chart, we have the form $dx_0 \wedge dx_1$.

$$dX \wedge dY = d(x_0)^2 \wedge d(x_0x_1) = 2x_0dx_0 \wedge (x_0dx_1 + x_1dx_0) = 2x_0^2dx_0 \wedge dx_1 = 2Xdx_0 \wedge dx_1$$

$$dx_0 \wedge dx_1 = \frac{dX \wedge dY}{2X} = \frac{du \wedge d(uv)}{2u} = \frac{du \wedge (udv + vdu)}{2u} = \frac{1}{2} du \wedge dv$$

The volume form remains well-behaved when approaching the exceptional divisor.

Weighted projective spaces

Example 2: $\mathbb{P}(1,1,1,2)$; the action is: $(x_0, x_1, x_2, x_3) \mapsto (tx_0, tx_1, tx_2, t^2x_3)$.

Describing orbits: if $x_3 \neq 0$, it contains $(?, ?, ?, 1)$. Stabilizer: $t^2 = 1$.

Residue action: $(x_0, x_1, x_2, x_3) \mapsto (-x_0, -x_1, -x_2, x_3)$

Weighted projective spaces

Example 2: $\mathbb{P}(1,1,1,2)$; the action is: $(x_0, x_1, x_2, x_3) \mapsto (tx_0, tx_1, tx_2, t^2x_3)$.

Describing orbits: if $x_3 \neq 0$, it contains $(?, ?, ?, 1)$. Stabilizer: $t^2 = 1$.

Residue action: $(x_0, x_1, x_2, x_3) \mapsto (-x_0, -x_1, -x_2, x_3)$

In the invariant chart $x_3 \neq 0$ it's: $(x_0, x_1) \rightarrow (x_0^2, x_0x_1, x_0x_2, x_1^2, x_2^2, x_1x_2) = (X, Y, Z, \dots)$

Weighted projective spaces

Example 2: $\mathbb{P}(1,1,1,2)$; the action is: $(x_0, x_1, x_2, x_3) \mapsto (tx_0, tx_1, tx_2, t^2x_3)$.

Describing orbits: if $x_3 \neq 0$, it contains $(?, ?, ?, 1)$. Stabilizer: $t^2 = 1$.

Residue action: $(x_0, x_1, x_2, x_3) \mapsto (-x_0, -x_1, -x_2, x_3)$

In the invariant chart $x_3 \neq 0$ it's: $(x_0, x_1) \rightarrow (x_0^2, x_0x_1, x_0x_2, x_1^2, x_2^2, x_1x_2) = (X, Y, Z, \dots)$

Resolution of singularities: $(u, v, w) \mapsto (u, uv, uw, uv^2, uw^2, uvw)$.

We get $(0,0,0, \dots)$ if $u = 0$ and v, w are arbitrary.

Weighted projective spaces

Example 2: $\mathbb{P}(1,1,1,2)$; the action is: $(x_0, x_1, x_2, x_3) \mapsto (tx_0, tx_1, tx_2, t^2x_3)$.

Describing orbits: if $x_3 \neq 0$, it contains $(?, ?, ?, 1)$. Stabilizer: $t^2 = 1$.

Residue action: $(x_0, x_1, x_2, x_3) \mapsto (-x_0, -x_1, -x_2, x_3)$

In the invariant chart $x_3 \neq 0$ it's: $(x_0, x_1) \rightarrow (x_0^2, x_0x_1, x_0x_2, x_1^2, x_2^2, x_1x_2) = (X, Y, Z, \dots)$

Resolution of singularities: $(u, v, w) \mapsto (u, uv, uw, uv^2, uw^2, uvw)$.

We get $(0,0,0, \dots)$ if $u = 0$ and v, w are arbitrary.

On the chart, we have the form $\omega = dx_0 \wedge dx_1 \wedge dx_2$. But it's not invariant. $\omega^{\otimes 2}$ is.

$$(dx_0 \wedge dx_1 \wedge dx_2)^{\otimes 2} = \frac{(dX \wedge dY \wedge dZ)^{\otimes 2}}{4X^3} = \frac{1}{4}u \cdot du \wedge dv \wedge dw$$

So, the volume shrinks when as approach the exceptional divisor.

Weighted projective spaces

Example 3: $\mathbb{P}(1,1,1,4)$; the action is: $(x_0, x_1, x_2, x_3) \mapsto (tx_0, tx_1, tx_2, t^4x_3)$.

In the chart $x_3 \neq 0$ it's: $(x_0, x_1) \rightarrow (x_0^4, x_0^3x_1, x_0^3x_2, \dots) = (X, Y, Z, \dots)$

Resolution of singularities: $(u, v, w) \mapsto (u, uv, uw, \dots)$.

We get $(0,0,0, \dots)$ if $u = 0$ and v, w are arbitrary.

On the chart, we have the form $\omega = dx_0 \wedge dx_1 \wedge dx_2$. But it's not invariant. $\omega^{\otimes 4}$ is.

$$(dx_0 \wedge dx_1 \wedge dx_2)^{\otimes 4} = \frac{(dX \wedge dY \wedge dZ)^{\otimes 4}}{256X^9} = \frac{1}{256} u^{-1} \cdot du \wedge dv \wedge dw$$

So, the volume explodes when as approach the exceptional divisor.

Terminal vs non-terminal singularities

- $\mathbb{P}(1,1,1,2)$ – volume shrinks, **terminal**
- $\mathbb{P}(1,1,2)$ – volume is well-behaved, **non-terminal** (canonical)
- $\mathbb{P}(1,1,1,4)$ – volume explodes, **non-terminal**

Picard-rank-2 toric variety

$X = (\mathbb{C}^n \setminus S)/(\mathbb{C}^\times)^2$, where $(t, s) \cdot z_i = t^{a_i} s^{b_i} z_i$ and S are the bad spots

Weight matrix: $\begin{pmatrix} a_1 & \cdots & a_n \\ b_1 & \cdots & b_n \end{pmatrix}$

Example 1. Matrix $\begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}$. Means $(t, s): (x_0, x_1, x_2, x_3) \mapsto (tx_0, tx_1, sx_2, sx_3)$.

Bad set: $\{x_0 = x_1 = 0\} \cup \{x_2 = x_3 = 0\}$.

The result is: $\mathbb{P}^1 \times \mathbb{P}^1$

Picard-rank-2 toric variety

Example 2. Matrix $\begin{pmatrix} 1 & 1 & 0 & -1 \\ 0 & 0 & 1 & 1 \end{pmatrix}$. Means $(t, s): (x_0, x_1, x_2, x_3) \mapsto (tx_0, tx_1, sx_2, t^{-1}sx_3)$.

Bad set: $S = \{x_0 = x_1 = 0\} \cup \{x_2 = x_3 = 0\}$. Weight (1, 0) Weight (0, 1)

Define $\Phi: \mathbb{C}^4 \setminus S \rightarrow \mathbb{P}^1 \times \mathbb{P}^2$, $(x_0, x_1, x_2, x_3) \mapsto ([x_0: x_1], [x_0x_3: x_1x_3: x_2])$

The image is $Z = \{([u_0: u_1], [v_0: v_1: v_2]) \mid u_0v_1 = u_1v_0 (= x_0x_1x_3)\}$

Let's now consider $Z \rightarrow \mathbb{P}^2$, $(U, V) \mapsto V$.

- For $V \neq (0: 0: 1)$, $(v_0, v_1) \neq (0, 0)$. Let $v_0 \neq 0 \Rightarrow u_1 = \frac{u_0v_1}{v_0}$ so there's unique pre-image of V in Z .
- For $V = (0, 0, 1)$, any U goes.

This is a **blow-up** of $\mathbb{P}^2$ in $(0: 0: 1)$.

Curves on varieties

Here, a **curve** means an embedding $\mathbb{P}^1 \rightarrow Z$

Example 1: $\mathbb{P}^2 \cong \mathbb{P}(1,1,1)$;

Take a point $[1:0:0] \in \mathbb{P}^2$. There is a 1-parametric family of curves passing through it:

$$f_{[b:c]}: \mathbb{P}^1 \rightarrow \mathbb{P}^2, \quad [w_0:w_1] \mapsto [w_0:-cw_1:bw_1]$$

A curve corresponding to $[b:c]$ is cut by a degree 1 equation $bu_1 + cu_2 = 0$

Curves on varieties

Example 2: $\mathbb{P}^1 \times \mathbb{P}^1$;

Take a point $([u_0:u_1], [v_0:v_1]) = ([1:1], [1:1]) \in \mathbb{P}^1 \times \mathbb{P}^1$. There are two special curves passing through it:

$$f_1: [w_0:w_1] \mapsto ([w_0:w_1], [1:1])$$

$$f_2: [w_0:w_1] \mapsto ([1:1], [w_0:w_1])$$

They are both cut by **linear** equations: $v_0 - v_1 = 0$ and $u_0 - u_1 = 0$ respectively.

And what about a “diagonal” curve

$$f_\Delta: [w_0:w_1] \mapsto ([w_0:w_1], [w_0:w_1])?$$

It's cut by a **quadratic** equation $u_0v_1 = v_0u_1$.

Curves on varieties

Example 3: Quadratic cone $Q = \{[X:Y:Z:W] \mid XZ - Y^2 = 0\} \cong \mathbb{P}(1,1,2)$;

A cone is swept by its generator lines that pass through the vertex $[0:0:0:1]$.

Take a point $[1:1:1:1]$. The generator line through it is

$$f: [w_0:w_1] \mapsto [w_0:w_0:w_0:w_1]$$

There is no other degree 1 curves passing through $[1:1:1:1]$.

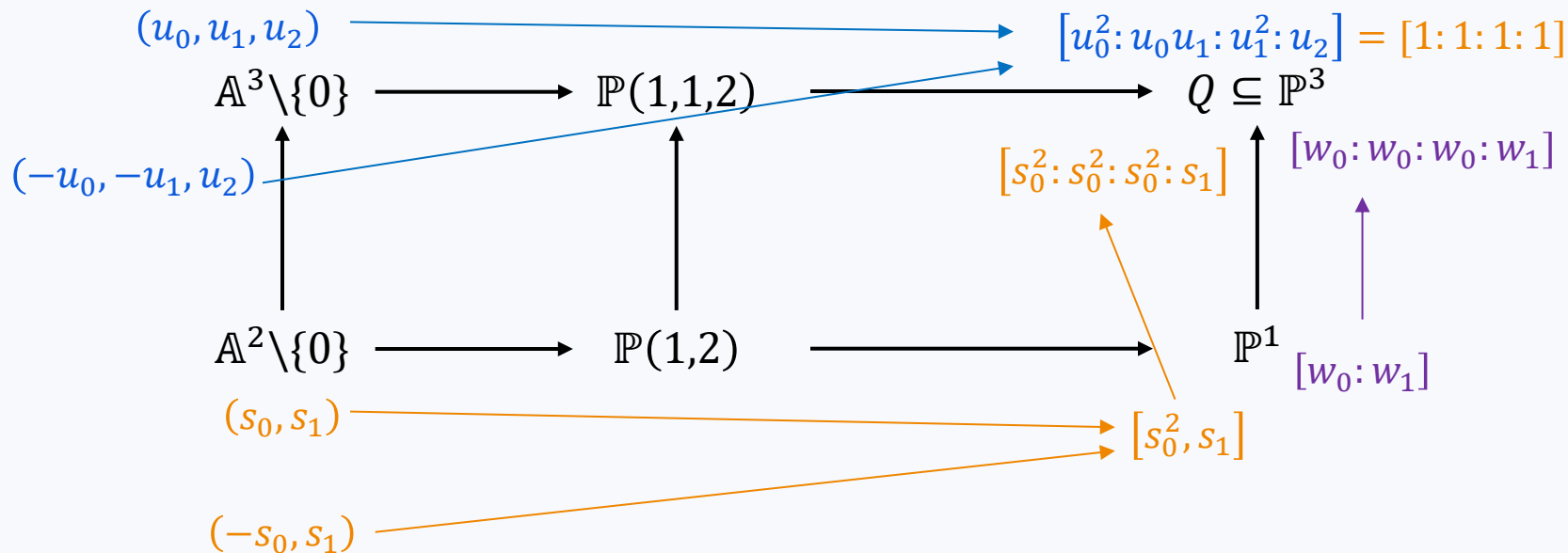
Some additional info here:

$$\begin{array}{ccccc} \mathbb{A}^3 \setminus \{0\} & \longrightarrow & \mathbb{P}(1,1,2) & \longrightarrow & Q \subseteq \mathbb{P}^3 \\ \uparrow & & \uparrow & & \uparrow \\ \mathbb{A}^2 \setminus \{0\} & \longrightarrow & \mathbb{P}(1,2) & \longrightarrow & \mathbb{P}^1 \end{array}$$

Curves on varieties

Example 3: Quadratic cone $Q = \{[X:Y:Z:W] \mid XZ - Y^2 = 0\} \cong \mathbb{P}(1,1,2)$;

Some additional info here:



Regularized quantum period

$$\hat{G}_X(t) = \sum_{d \geq 0} c_d t^d, \quad c_d = r_d d!$$

where r_d are funny numbers connected to curve counting and moduli spaces.

$$\hat{G}_{\mathbb{P}^2}(t) = 1 + 6t^3 + 90t^6 + 1680t^9 + \dots \quad 3 = 1 + 1 + 1, \quad 6 = \mathbf{1} \cdot 3!$$

$$\hat{G}_{\mathbb{P}^1 \times \mathbb{P}^1}(t) = 1 + 4t^2 + 36t^4 + 400t^6 + \dots, \quad 4 = \mathbf{2} \cdot 2!$$

$$\hat{G}_{cone}(t) = 1 + 12t^4 + 420t^8 + 18480t^{12} + \dots, \quad 4 = 1 + 1 + 2, \quad 12 = \frac{\mathbf{1}}{\mathbf{2}} \cdot 4!$$

Regularized quantum period

$$\hat{G}_X(t) = \sum_{d \geq 0} c_d t^d, \quad c_d = r_d d!$$

where r_d are funny numbers connected to curve counting and moduli spaces.

For $\mathbb{P}(a_0, \dots, a_n)$, $a = a_0 + \dots + a_n$:

$$\hat{G}_X(t) = \sum_{k \geq 0} \frac{(ak)!}{(a_1 k)! \cdot (a_2 k)! \cdot \dots \cdot (a_n k)!} t^{ak}$$

For toric varieties of Picard rank two with a_i, b_i , $a = a_1 + \dots + a_n$, $b = b_1 + \dots + b_n$:

$$\hat{G}_X(t) = \sum_{(k,l) \in \mathbb{Z}^2 \cap C} \frac{(ak + bl)!}{(a_1 k + b_1 l)! \cdot \dots \cdot (a_n k + b_n l)!} t^{ak + bl}$$

$$C = \{(x, y) \in \mathbb{R}^2 \mid a_i x + b_i y \geq 0, \forall i\}$$

Machine Learning the Dimension of a Fano Variety

Data:

- **150,000 weighted projective spaces** with terminal singularities and $\dim \leq 10$.
 - took $c_d \neq 0$, fitted a linear model to $(d, \log c_d)$,
 - used **slope and intercept**. On 10%+ of 149,998 examples: **linear SVM** with $C = 10$, after standardization $\rightarrow$ **99.99% accuracy**.
- **200,000 toric varieties of Picard rank two** with terminal singularities and $\dim \leq 10$, generated from random weight matrices with entries between 0 and 5 and deduplicated by fan isomorphism.
 - Also took slope&intercept for $(d, \log c_d) +$ some c_d coefficients
 - **87.7% (SVM), 88.6% (Random Forest), 88.7% (MLP2), and 97.7% (MLP102)**

Machine Learning the Dimension of a Fano Variety

Interpretation:

- Computed **SHAP** values and saw that slope & intercept are more important than specific values of c_p .
- Established and proved the following asymptotic for **weighted projective spaces** and **toric varieties of Picard rank two**:

$$\log c_d \sim Ad + \frac{\dim X}{2} \log d + B$$

A and B depend on the variety. For example, for $\mathbb{P}(a_0, \dots, a_n)$, $p_i := \frac{a_i}{a}$

$$A = - \sum_i p_i \log p_i, \quad B = - \frac{\dim X}{2} \log(2\pi) - \frac{1}{2} \sum_i \log p_i$$

Machine Learning Detects Terminal Singularities

- **10M million** examples, balanced between **terminal** and **non-terminal** singularities. Each example – a **2×10 weight matrix** in standard form + some reasonable constraints. (Cost 150 CPU years...)
- Trained an **MLP** on the weights to predict terminal/non-terminal. Tuned hyperparameters thoroughly.
- **95% accuracy** on weights bounded by 7.
- **Out-of-sample generalization is poor**. Accuracy drops to **62%** for entries bounded by 8, **52%** for bound 9, and **50%** for bound 10.
- Used it to quickly produce 100M Fano with terminal singularities candidates

Inspiration

-> theory

Proposition 3. Consider a weight matrix

$$\begin{bmatrix} a_1 & \cdots & a_N \\ b_1 & \cdots & b_N \end{bmatrix}$$

that satisfies conditions (1)–(4) from §3. Let $g_i = \gcd\{a_i, b_i\}$, and let A_i, B_i be integers such that $A_i a_i + B_i b_i = g_i$. Set

$$\alpha_i^j = \frac{a_j b_i - b_j a_i}{g_i}$$

$$\alpha_i = \sum_{j=1}^N \alpha_i^j$$

$$\beta_i^j = -A_i a_j - B_i b_j$$

$$\beta_i = \sum_{j=1}^N \beta_i^j$$

$$f_i = \frac{\alpha_i g_i}{\gcd\{g_i, \beta_i\}}$$

noting that all these quantities are integers. Then Δ_i is mostly empty if and only if for all $k \in \{0, \dots, f_i - 1\}$ and $l \in \{0, \dots, g_i - 1\}$ such that

$$\sum_{j=1}^N \left\{ k \frac{\alpha_i^j}{f_i} + l \frac{\beta_i^j}{g_i} \right\} = 1$$

we have that

$$\left\{ k \frac{\alpha_i^j}{f_i} + l \frac{\beta_i^j}{g_i} \right\} = \left\{ \frac{\alpha_i^j}{\alpha_i} \right\}$$

for all j .

Inspiration

->

theory

Algorithm 1 Test terminality for weight matrix $W = [[a_1, \dots, a_N], [b_1, \dots, b_N]]$.

- 1: Set $a = \sum_{i=1}^N a_i$, $b = \sum_{i=1}^N b_i$.
- 2: Set $s_+ = \{i \mid a_i b - b_i a > 0\}$ and $s_- = \{i \mid a_i b - b_i a < 0\}$.
- 3: Set I to be the smaller of s_+ and s_- .
- 4: **for** $i \in I$ **do**
- 5: Test if Δ_i is mostly empty, using Proposition 3.
- 6: **if** Δ_i is not mostly empty **then**
- 7: return False.
- 8: **end if**
- 9: **end for**
- 10: return True.

Recap

In this class we have learned:

- Logistic Regression → Fully-Connected Neural Network
- Training of neural networks, idea of backpropagation
- Solving binary classification and regression tasks with neural networks
- Variants of activation functions
- Building neural network on PyTorch